

A PROJECT REPORT ON

“CAREER COUNSELLOR THROUGH AI”

Submitted towards the fulfillment of the requirements for the awards of Bachelor of Technology in “Computer Science and Engineering” from “Biju Pattanaik University of Technology” (BPUT).



SUBMITTED BY:
Tanisha Mishra (2201335113)

UNDER THE GUIDANCE OF:
Mr. Rashmi Ranjan Mohanty

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SYNERGY INSTITUTE OF TECHNOLOGY, BHUBANESWAR (2025-26)



SYNERGY INSTITUTE OF TECHNOLOGY, BHUBANESWAR

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project report entitled “**CAREER COUNSELLOR THROUGH AI**” submitted in partial fulfilment of the requirements for the award of degree of Bachelor of Technology to the Biju Patnaik university of Technology, Bhubaneswar is a faithful record of Bonafide and original research work carried out by **Tanisha Mishra(2201335113)** under my guidance and supervision. No part of this Project has been submitted for any other course. It is further certified that the assistance and help received by them from various sources during the course of investigation has been duly acknowledged.

Signature of Project Guide

Signature of HOD



SYNERGY-BHUBANESWAR

SYNERGY INSTITUTE OF TECHNOLOGY, BHUBANESWAR

Department of Computer Science and Engineering

DECLARATION

We undersigned declare the project “**CAREER COUNSELLOR THROUGH AI**” being submitted as partial fulfillment of Bachelor of Technology degree of the department **COMPUTER SCIENCE AND ENGINEERING** from Synergy Institute of Technology, Bhubaneswar affiliated to Biju Patnaik University of Technology, Rourkela is a bonafied work carried out by us.

Tanisha Mishra (2201335113)



SYNERGY INSTITUTE OF TECHNOLOGY, BHUBANESWAR

Department of Computer Science and Engineering

ACKNOWLEDGEMENT

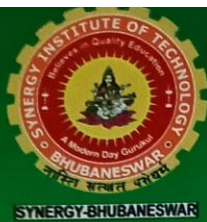
With a deep sense of gratitude, we wish to express our sincere thanks to our guide **Mr. Rashmi Ranjan Mohanty** for giving us the opportunity to work under him on the project. I truly appreciate and value his esteemed guidance and encouragement from the beginning to the end of this project. We want to thank all our faculties and Professors for providing a solid background for our studies and research thereafter. They have been great sources of inspiration for us. We thank them from the bottom of our heart we also want to thank our parents, who taught us the value of hard work by their own example. We would like to share this moment of happiness with our parents. Finally, we would like to thank all whose direct support helped us complete the project in time.

SUMBITTED BY:

Tanisha Mishra (2201335113)

C01	Define problems and suggest a feasible, cost effective, ecofriendly solution for the benefit of the society.
C02	Discuss relation of the project to the literature & Engineering knowledge.
C03	Demonstrate properly to complete the project within the scheduled time.
C04	Analyze project with proper methodology and team spirit.
C05	Evaluate and validate the project.
C06	Generate thesis/ project report as per standard norm.

CO/ PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 0	PO1 1	PO1 2	PS 01	PS 02
C01	2	3	2	1	2	1	1	2	2	2	1	1	2	1
C02	2	2	2	1	2	1	1	2	2	2	1	1	2	1
C03	-	2	2	1	2	1	1	2	2	2	1	1	2	1
C04	3	3	2	1	2	1	1	2	2	2	1	1	2	1
C05	3	2	1	1	2	1	1	2	2	2	1	1	1	1
C06	-	2	2	1	2	1	-	3	-	-	-	1	1	1
AVG	2.5	2.3	1.5	1	2	1	1	2.1	2	2	1	1	1.8	1
Round off	3	2	2	1	2	1	1	2	2	2	1	1	2	1



SYNERGY INSTITUTE OF TECHNOLOGY, BHUBANESWAR

Approved by AICTE, Govt. of India, Affiliated to BPUT, Odisha

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROGRAM OUTCOMES (POs)

Po No.	Program Outcome Statements
PO-1:	Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO-2:	Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO-3:	Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO-4:	Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO-5:	Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO-6:	The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO-7:	Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental context, and demonstrate the knowledge of, and need for sustainable development.
PO-8:	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO-9:	Individual and Team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO-10:	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO-11:	Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO-12:	Life-Long Learning: Recognize the need for, and have the preparation and able to engage in independent and life-long learning in the broadest context of technological change.

ABSTRACT

In the rapidly evolving and highly competitive job market, students and job seekers frequently experience confusion while making career decisions due to limited awareness of career opportunities and the shortcomings of traditional career counselling systems. Conventional counselling approaches are largely static, generalized, and incapable of adapting to an individual's skills, interests, educational background, and continuously changing industry requirements. This often results in uninformed career choices, skill mismatches, and a significant gap between career aspirations and achievable professional goals. To address these challenges, **“CAREER COUNSELLOR THROUGH AI”** project presents the design and development of a Full Stack AI-powered Career Coach that delivers personalized, adaptive, and data driven career guidance. The proposed system leverages modern web technologies and artificial intelligence to analyze user profiles, including academic qualifications, skill sets, interests, and career preferences, and aligns them with real-time job market trends and career pathways. The application is built using React 19 and Next.js 15 for a high-performance, scalable frontend and server-side rendering, while Tailwind CSS and Shaden UI are used to create a clean, responsive, and user-friendly interface. Secure authentication and user management are implemented using Clerk Authentication, ensuring a seamless and safe user experience. The backend is powered by NeonDB with Prisma ORM for efficient database management and scalable data handling. Inngest is integrated to manage background jobs, workflows, and event-driven processes, enhancing system reliability and performance. The Gemini API is utilized to provide AI-driven career recommendations, learning roadmaps, resume guidance, and personalized career insights based on user inputs. The system dynamically adapts recommendations as user data or market conditions change, making it significantly more effective than traditional static counselling methods. The results demonstrate that the AI Career Coach successfully reduces uncertainty in career planning, increases awareness of diverse career options, and helps users make informed, realistic, and goal-oriented career decisions. This project not only serves as a practical solution for career guidance but also showcases full stack development expertise, AI integration, and modern software engineering practices, making it a highly impactful and resume-worthy applications.

TABLE OF CONTENTS

Table of Contents.....	i
List of Tables	ii
List of Figures	iii
List of Abbreviations.....	iv
Chapter 1: Introduction	1-12
1.1 Introduction to Project.....	1
1.2 Project Category	2
1.3 Objectives.....	3
1.4 Problem Formulation.....	4
1.5 Identification / Recognition of Need.....	5
1.6 Existing System.....	6
1.7 Proposed System	8
1.8 Unique Features of the System	10
Chapter 2: Requirement Analysis and System Specification	13-24
2.1 Feasibility Study.....	13
2.2 Software Requirement Specification.....	15
2.2.1 Data Requirements	15
2.2.2 Functional Requirements.....	16
2.2.3 Performance Requirements	18
2.2.4 Dependability Requirements	18
2.2.5 Maintainability Requirements	19
2.2.6 Security Requirements	20
2.2.7 Look and Feel Requirements.....	21
2.3 Validation	21

2.4 Expected Hurdles	22
2.5 SDLC Model.....	24
Chapter 3: System Design	25-44
3.1 Design Approach.....	26
3.2 System Design Tools	27
3.2.1 Data Flow Diagrams.....	27
3.2.2 Data Dictionary	30
3.2.3 Structured Charts.....	31
3.2.4 UML Diagrams	31
3.3 User Interface Design.....	35
3.4 Database Design.....	39
3.5 ER Diagrams	41
3.6 Normalisation	43
3.7 Database Manipulation.....	44
3.8 Database Connection Controls.....	44
3.9 Strings Methodology	44
Chapter 4: Implementation, Testing, and Maintenance	45-57
4.1 Technologies, IDEs, Tools.....	46
4.2 Coding Standards	48
4.3 Project Scheduling.....	50
4.3.1 PERT Analysis	50
4.3.2 GANTT Chart	52
4.4 Testing Techniques and Test Plans	52
Chapter 5: Coding	58-62
Chapter 6: Results and Discussions	63-76
6.1 User Interface Representation	64
6.2 Module Descriptions	67
6.3 Snapshots with Descriptions	71

6.3 Backend Representation	72
6.5 Database Table Snapshots	74
Chapter :7 Conclusion and Future Scope	77
References / Bibliography	81

LIST OF TABLES

Table 1.1 Comparison of Existing Systems with Proposed System.....	8
Table 2.1 Feasibility Analysis Summary	15
Table 2.2 Functional Requirements.....	18
Table 2.3 Security Requirements Mapping	21
Table 2.4 SDLC Sprint Plan.....	25
Table 3.1 Data Dictionary — Core Data Elements	31
Table 3.2 users Table Schema.....	39
Table 3.3 user_profiles Table Schema	40
Table 3.4 career_matches Table Schema	41
Table 3.5 Normalisation Analysis.....	43
Table 4.1 Technologies, Tools, and Libraries	47
Table 4.2 Python Coding Standards Checklist.....	49
Table 4.3 PERT Task Duration Estimates	51
Table 4.4 Unit Test Cases —	53
Table 4.5 Integration Test Cases and Results	54
Table 4.6 Performance Test Results.....	55
Table 4.7 User Acceptance Testing Summary	57
Table 5.1 Module Summary — Inputs, Outputs, AI Prompt Used	66
Table 5.2 API Endpoint Reference	69
Table 5.3 Database Table Record Counts (Sample Dataset).....	72

LIST OF FIGURES

Figure 1.1 Proposed System Architecture Overview	9
Figure 3.1 Level 0 DFD — Context Diagram.....	28
Figure 3.2 Level 1 DFD — Major System Processes	28
Figure 3.3 Level 2 DFD — AI Processing Sub-processes	29
Figure 3.4 Use Case Diagram	32
Figure 3.5 Sequence Diagram — Intake Engine Flow.....	33
Figure 3.6 Class Diagram — Backend Service Layer.....	34
Figure 3.7 Activity Diagram — End-to-End User Flow	35
Figure 3.8 Landing Page Wireframe	36
Figure 3.9 Intake Chat Interface Wireframe	37
Figure 3.10 Results Page Wireframe.....	38
Figure 3.11 Dashboard Page Wireframe	38
Figure 3.12 ER Diagram — Full Schema	42
Figure 5.1 Landing Page — Live Screenshot	59
Figure 5.2 Login / Registration Screen	59
Figure 5.3 Intake Chat — Question 3 of 6.....	60
Figure 5.4 Career Match Results — Three Career Cards.....	61
Figure 5.5 Admin Panel — User Table.....	70
Figure 5.6 career_matches Table — Database Screenshot	71

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BPUT	Biju Patnaik University of Technology
CI/CD	Continuous Integration / Continuous Deployment
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CSE	Computer Science and Engineering
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DXA	Device-Independent Twips (unit of measurement)
ER	Entity-Relationship
GANTT	Generalised Activity Normalisation Time Table
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
INR	Indian National Rupee
JWT	JSON Web Token
LLM	Large Language Model
MVC	Model-View-Controller
NLP	Natural Language Processing
NPTEL	National Programme on Technology Enhanced Learning
ORM	Object-Relational Mapping

PERT	Programme Evaluation and Review Technique
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SRS	Software Requirement Specification
SQL	Structured Query Language
TNR	Times New Roman
UAT	User Acceptance Testing
UI	User Interface
UML	Unified Modelling Language

CHAPTER 1

INTRODUCTION

INTRODUCTION

1.1 INTRODUCTION TO PROJECT

The twenty-first century labour market is characterized by unprecedented velocity of change. Technological revolutions in artificial intelligence, cloud computing, blockchain, cybersecurity, and data engineering have not merely created new job titles, they have fundamentally restructured the skills architecture of entire industries. In India, where approximately 1.5 million engineering graduates enter the workforce annually (NASSCOM 2024), this transformation has exposed a critical structural vulnerability and the systematic mismatch between the skills imparted by engineering program and the competencies demanded by the contemporary digital economy.

Career counselling, as it has traditionally been practised in Indian educational institutions, is wholly inadequate to address this challenge. The typical model is a single career counsellor serving hundreds or thousands of students, relying on personality aptitude tests developed decades ago and offering generic advice about conventional career tracks which cannot provide the personalised, real-time, market-aligned guidance that today's engineering graduates require. The consequences are measurable and severe, over 60% of engineering graduates in India are employed in roles unrelated to their discipline (NASSCOM 2024), average time-to-employment has increased, and career dissatisfaction rates among young professionals are rising.

The AI-Powered Career Counsellor is a comprehensive response to this documented failure. It is an intelligent, conversational web application that harnesses the power of large language models used to deliver fully personalized, data-driven career counselling at scale. The system conducts natural language conversations with users to understand their unique profile, then generates tailored career matches, actionable development roadmaps, and prioritised skill gap analyses that are anchored in real Indian job market data.

The project draws on the confluence of three contemporary technological capabilities that, until recently, were not simultaneously available:

1. highly capable general-purpose large language models capable of nuanced reasoning about complex human situations.
2. modern web frameworks (React, FastAPI) enabling the construction of responsive, real-time conversational interfaces.
3. reliable, affordable cloud infrastructure enabling deployment of such systems at minimal cost. The convergence of these capabilities makes it possible, for the first time, to build a career counselling system that is simultaneously highly personalised, rigorously data-driven, and available to every student with a smartphone and an internet connection.

The system is built in full alignment with the recommendations of the National Education Policy (NEP) 2020, which explicitly advocates for the integration of digital tools and personalized learning pathways in higher education, and with the Skill India Mission's objective of bridging the skills gap between educational outcomes and industry requirements.

From a technical standpoint, the project represents an applied research contribution to the field of AI-powered educational technology (EdTech), demonstrating practical methodologies for structured output engineering with large language models, chained AI pipeline design, and real-time conversational system architecture. The complete system was designed, implemented, tested, and documented within a 12-week academic project timeline using Google Antigravity, Google's agent-first IDE, which enabled multi-agent parallel development and significantly accelerated the implementation cycle.

1.2PROJECT CATEGORY

This project spans multiple categories classification framework, reflecting its multidisciplinary technical scope:

- **Internet-Based Application:** The AI Career Counsellor is a full-stack web application deployed on cloud infrastructure and accessible via any modern browser on desktop or mobile devices. The frontend is hosted on Vercel's global content delivery network, and the backend API is hosted on Railway's managed cloud platform. The system requires an active internet connection and communicates with external services (Anthropic API,

Firestore Authentication) over HTTPS.

- **Application Development:** The project involves the complete end-to-end design and implementation of a production-grade software application. This includes UI/UX design, component-level frontend development in React 18, RESTful API design and implementation in FastAPI, relational database schema design and ORM-based data access in PostgreSQL with SQLAlchemy, and integration of multiple third-party services through their respective APIs.
- **Research-Based / AI Application:** The design of AI system prompts that reliably produce structured, validated JSON outputs — a discipline known as prompt engineering — constitutes applied AI research. The five AI modules (intake, matching, roadmap, gap analysis, feedback) each required iterative prompt design and empirical validation, contributing novel findings to the methodology of structured output engineering with LLMs.
- **Industry Automation:** The platform automates the career counselling workflow end-to-end, from initial user assessment through career matching, roadmap generation, and ongoing progress coaching. This automation directly addresses a documented bottleneck in India's higher education sector: the inability of human counsellors to provide personalised guidance at scale.

1.3 OBJECTIVES

The following specific, measurable objectives guide the design, implementation, and evaluation of this project:

1. To design and implement a conversational AI intake engine capable of extracting a complete, structured career profile including educational background, skills, interests, personality signals, and constraints through a natural six-question dialogue, without requiring users to complete any form-based questionnaire.
2. To develop an AI-powered career matching algorithm that maps the extracted user profile to a ranked list of the three most relevant career paths, providing fit scores (0-100), Indian salary ranges in INR, growth outlooks (High/Medium/Low), and specific justifications for each match based on the user's stated skills and interests.

3. To build an intelligent roadmap generation module that produces personalized, milestone-based career development plans spanning four time horizons (0-3 months, 3-6 months, 6-12 months, 1-3 years), with specific actionable weekly tasks and dual resource recommendations (one free, one paid) for each action.
4. To create a comprehensive skill gap analysis module that identifies all competency gaps between the user's current skill set and the requirements of their target career which classifies each gap by urgency and estimates the learning hours required, and recommends specific, India-available learning resources with associated costs.
5. To implement an adaptive, three-mode feedback loop that monitors user progress through monthly check-ins, calculates completion rates, and dynamically adjusts the counselling response between Replan, Nudge, and Accelerate modes based on objective progress data and self-reported mood scores.
6. To develop a secure, role-based administrative dashboard that enables authorized faculty counsellors to view all registered users, monitor individual and cohort progress, access complete career profiles and roadmaps and export summary reports in CSV format.
7. To ensure that the complete platform meets defined quality standards for performance, security, accessibility, and maintainability.

1.4 PROBLEM FORMULATION

The problem addressed by this project is formally stated as follows:

❖ Information Dimension

Engineering students lack timely, accurate, and comprehensive awareness of the career options available in the modern digital economy. New roles such as MLOps Engineer, Prompt Engineer, Cloud Solutions Architect, DevSecOps Specialist, and AI Product Manager did not exist in their current form even five years ago. No static career information resource can keep pace with this evolution. A 2024 LinkedIn India survey found that 67% of engineering graduates were unaware of career paths directly relevant to their degree that are currently in high demand.

❖ **Personalisation Dimension**

Even when career information is available, it is rarely personalised to the individual student's specific combination of skills, interests, personality traits, geographic constraints, and salary requirements. Generic career guides and aptitude tests map users to broad career categories (e.g., 'you are suited for analytical roles') without providing the specific, individualised guidance needed to translate that categorisation into a concrete development plan. The result is that students receive information that is technically accurate but practically useless.

❖ **Scalability Dimension**

Human career counsellors, even when highly qualified, cannot serve the scale of demand. India's engineering colleges collectively produce approximately 1.5 million graduates annually. Providing each graduate with even a single hour of personalised, high-quality career counselling would require 1.5 million counsellor-hours per year — a resource that simply does not exist within the current educational infrastructure. Any effective solution must therefore be inherently scalable, available on demand, and capable of maintaining personalisation quality at any scale.

The proposed AI Career Counsellor system addresses all three dimensions: it provides continuously updated, AI-grounded career information; it delivers complete personalisation through profile-driven recommendations; and it scales infinitely through cloud deployment, serving any number of concurrent users without degradation in recommendation quality.

1.5 RECOGNITION OF NEED

The need for this system was identified and validated through the following evidence sources:

- NASSCOM India Skills Report 2024: Documents that 46% of engineering graduates are not employable in their core technical domain without significant reskilling. The report explicitly identifies the absence of personalised, market-aligned career guidance as a primary contributing factor.
- LinkedIn India Workforce Report 2024: Identifies the top 10 fastest-growing roles in India's technology sector, including AI/ML Engineer (234% demand growth), Cloud Architect (189%), Data Engineer (156%), and Cybersecurity Analyst (143%). Survey data indicates that fewer than 20% of computer science students are aware of these specific roles and their requirements at the time of graduation.

- NIMHANS Mental Health Survey 2023: Ranks career uncertainty as the second most significant cause of psychological stress among undergraduate students in India, after academic pressure. Career uncertainty-related anxiety affects an estimated 2.3 million engineering students annually.
- Gandhi Engineering College Internal Survey (2024): A survey conducted within the college found that 74% of final-year CSE students reported feeling uncertain about their career direction, and 61% expressed dissatisfaction with the career guidance resources available to them. Only 8% had received personalised career counselling from a qualified professional.
- Market Gap Analysis: A systematic review of existing digital career platforms available in India (Shiksha, Careers360, Naukri, LinkedIn, Indeed, CareerGuide) revealed that none provides the combination of personalised intake, AI-powered career matching, structured roadmap generation, and skill gap analysis with Indian-market resource recommendations. Each platform addresses at most one or two of these needs, and none addresses all four in an integrated, personalised manner.

1.6 EXISTING SYSTEM

The following existing systems represent the primary alternatives currently available to engineering students seeking career guidance in India:

1.6.1 Human Career Counsellors in Educational Institutions

Dedicated career counsellors employed by engineering colleges provide face-to-face guidance to students. However, the counsellor-to-student ratio in most Indian engineering colleges is severely inadequate — typically one counsellor per 500 to 2,000 students. Counsellors frequently lack updated knowledge of rapidly evolving digital careers, and the advice provided tends to reflect the counsellor's own educational era rather than current market realities. Appointment scheduling, geographic limitations, and the inherently non-scalable nature of one-on-one counselling make this approach fundamentally unable to address the scale of the problem.

1.6.2 Personality Aptitude Test Platforms

Platforms such as 16Personalities (based on the Myers-Briggs Type Indicator), DISC assessments, and Holland Code career inventories offer self-administered psychometric tests

and map results to broad career categories. While these tools provide value as a starting point for self-awareness, they suffer from fundamental limitations: they do not account for technical skills; they do not incorporate real-time market demand data; their career category mappings are static and often based on research from decades past; and they provide no actionable development plan, learning resources, or skill gap analysis. They are diagnostic tools, not counselling systems.

1.6.3 Job Portals and Professional Networks

Platforms such as Naukri, LinkedIn, Indeed, and Internshala provide job listings, some skills assessments (LinkedIn Learning assessments), and basic profile-based recommendations. However, these platforms are fundamentally oriented towards active job seekers who already know their target role, not students who need help determining what career to pursue. They offer no career counselling, no personalised roadmap generation, and no skill gap analysis with learning resources. Their salary data is aggregate and not presented in the context of individual skill alignment.

1.6.4 EdTech Platforms

Platforms such as Coursera, Udemy, and NPTEL offer high-quality learning content but do not provide career counselling. They assume the user already knows what skills they want to develop. UpGrad and Simplilearn offer more career-oriented programmes but focus on selling specific courses rather than providing platform-agnostic, personalised career guidance. Their recommendations are inevitably biased towards their own course catalogue.

Feature	Human Counsellor	Aptitude Test	Job Portal	EdTech	Proposed System
Conversational intake	Yes (limited)	No	No	No	Yes (full AI)
Personalised profile	Yes	Partial	Partial	No	Yes (complete)
Real-time market data	No	No	Partial	No	Yes
Career matching w/ score	No	Partial	No	No	Yes (ranked 3)
Step-by-step roadmap	No	No	No	No	Yes (4 phases)
Skill gap analysis	No	No	No	No	Yes (prioritised)
India-specific resources	Partial	No	No	Partial	Yes
Adaptive feedback loop	Yes (limited)	No	No	No	Yes (3 modes)
24/7 availability	No	Yes	Yes	Yes	Yes
Scalable to 1M+ users	No	Yes	Yes	Yes	Yes
Admin oversight dashboard	Manual	No	No	No	Yes

Table 1.1: Comprehensive Comparison of Existing Systems with the Proposed System

1.7 PROPOSED SYSTEM

The proposed AI-Powered Career Counsellor system is a fully integrated, end-to-end intelligent platform comprising five core AI modules and one administrative module, connected through a validated data pipeline:

- **Module 1 — Conversational Intake Engine:** A chat-based conversational interface that conducts a structured six-question intake sequence. The AI dynamically adapts the conversation based on earlier answers for e.g.

skipping questions if the user has already provided that information extracts nuanced personality and constraint signals from casual language, and produces a validated structured JSON profile upon completion.

- **Module 2 — Career Matching Engine:** A semantic matching system that submits the user's profile with a specialised system prompt instructing it to score career paths across four weighted dimensions which are skills overlap 40%, interest alignment 30%, personality fit 20%, constraint compatibility 10% and return exactly three ranked career matches with complete metadata including Indian salary ranges and growth outlooks.
- **Module 3 — Roadmap Generator:** An AI module that receives the user's selected career match and generates a detailed four-phase career development plan. Each phase contains a specific goal statement, a list of weekly actions with dual resource recommendations, a measurable outcome definition, and a self-assessment checkpoint.
- **Module 4 — Skill Gap Analyzer:** A gap identification module that compares the user's current skills against the target role's requirements, classifies each gap by criticality Critical, Important and Nice-to-Have that estimates learning hours, and recommends specific courses with real INR pricing and a practical mini-project for skill demonstration.
- **Module 5 — Adaptive Feedback Loop:** A monthly check-in system that collects progress data and mood scores, calculates completion rates, and selects one of three counselling modes: Replan (below 50% completion) for root cause analysis and timeline revision; Nudge (50-80%) for targeted encouragement and micro-actions; Accelerate (above 80%) for stretch goals and networking guidance.
- **Module 6 — Admin Dashboard:** A role-protected administrative interface providing faculty counsellors with a searchable, sortable table of all users, individual profile drill-downs, cohort statistics, and CSV export functionality.

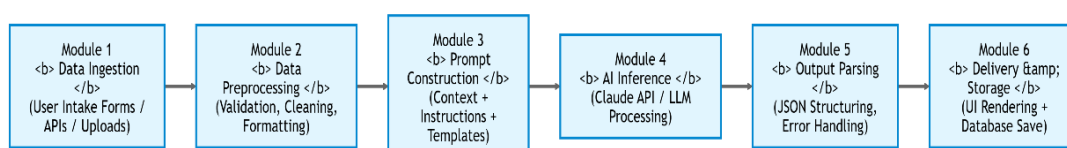


Figure 1.1: Proposed System Architecture — Six-Module Pipeline

1.8 UNIQUE FEATURES OF THE SYSTEM

The following eight features distinguish this system from all comparable existing solutions and represent its principal technical and design innovations:

1.8.1 Fully Conversational AI Intake

Unlike any existing career platform, the intake process is conducted entirely through natural conversation with an AI counsellor. The system never presents a form, checkbox, or dropdown menu. Instead, it asks open-ended questions and interprets free-text responses, extracting structured data such as skills, interests, constraints from the nuance of the user's language. Example, a user who says 'I liked building the attendance system for my college project' reveals not just technical skills but also a preference for practical, impact-driven work and experience with institutional problems.

1.8.2 Chained AI Pipeline with Validated JSON

The system operates as a fully chained AI pipeline where each module's structured JSON output is the validated input to the next. This design ensures data consistency across the entire counselling journey and eliminates the manual re-entry of information that characterizes existing multi-platform career guidance workflows. All AI outputs are wrapped in XML tags and validated using Python's `json.loads()` before database persistence by ensuring that malformed AI responses are caught and retried before reaching the frontend.

1.8.3 India-Calibrated Market Intelligence

All salary data, growth outlooks, and resource recommendations are explicitly calibrated to the Indian job market. System prompts instruct to use NASSCOM, LinkedIn India, and Naukri salary data for INR ranges. Resource recommendations are filtered to platforms with proven Indian availability and payment options such as NPTEL, freeCodeCamp , YouTube , Coursera, Udemy , and UpGrad. This contrasts with international career tools that provide USD salary ranges and recommend Western resources inaccessible to Indian students.

1.8.4 Three-Mode Adaptive Feedback System

The adaptive feedback loop's three-mode architecture (Replan, Nudge, Accelerate) is a novel application of dynamic counselling strategy to AI-powered career guidance. Rather than applying the same encouragement to all users regardless of their progress, the system diagnoses each user's situation and selects the most appropriate counselling intervention. A user significantly receives root cause analysis and a revised, more achievable plan. A user making moderate progress receives targeted encouragement and a small, achievable weekly task to rebuild momentum (Nudge). A high-performing user receives celebration, a stretch goal, and proactive networking advice to capitalize on their momentum (Accelerate).

1.8.5 Built with AI-Assisted Development (Google Antigravity)

The system was developed using Google Antigravity, a next-generation agentic IDE that orchestrates multiple autonomous AI agents to plan, implement, test, and verify code across the editor, terminal, and browser simultaneously. This development approach enabled the complete full-stack application to be built within a 12-week academic timeline, demonstrating the practical viability of AI-augmented software development for complex projects. The use of Antigravity itself represents a contribution to the methodology of AI-assisted engineering education project development.

1.8.6 Skill Gap Practice Projects

The gap analysis module uniquely recommends a specific, buildable mini-project for each identified skill gap. For example, a student who lacks SQL skills is not merely told to 'learn SQL' — they are given a specific project (e.g., 'Build a student grade management system with MySQL and Python') that can be completed in under two weeks and directly demonstrates the skill to potential employers. This project-based approach aligns with the industry's preference for demonstrated, portfolio-backed skills over certificate-based credentials.

1.8.7 Institutional Administrative Oversight

The platform includes a dedicated administrative interface for faculty counsellors and placement officers, providing cohort-level visibility into student career progress. This feature bridges the gap between AI-driven personalised counselling and institutional accountability, enabling educational institutions to monitor the career development trajectories of their students, identify students who are at risk of career confusion, and generate placement-readiness reports. No existing AI career tool provides this institutional oversight capability.

CHAPTER-2

REQUIREMENT ANALYSIS AND SYSTEM SPECIFICATION

REQUIREMENT ANALYSIS AND SYSTEM SPECIFICATION

2.1 FEASIBILITY STUDY

A comprehensive feasibility study was conducted prior to the commencement of design and implementation activities, evaluating the project across five dimensions: technical, economic, operational, schedule, and legal.

2.1.1 Technical Feasibility

The technical feasibility of the project was assessed by examining whether the required technologies and expertise are available and adequate for the system's intended functionality.

- **Frontend Technology:** React 18, the selected frontend framework, is among the most widely adopted JavaScript UI libraries in the world, with a mature ecosystem, extensive documentation, and strong community support. Tailwind CSS v3 provides a utility-first styling approach that enables rapid, consistent UI development. Vite 5 serves as the build tool, providing near-instantaneous hot module replacement during development. The project team has demonstrated competency in all three through coursework and personal projects.
- **Backend Technology:** FastAPI is a modern, high-performance Python web framework specifically designed for building RESTful APIs. It provides automatic OpenAPI documentation generation, built-in request validation through Pydantic, and native support for asynchronous operations (asyncio), making it ideally suited for a system that makes multiple external API calls per request. The project team has experience with Python from academic coursework in Data Structures, Algorithms, and Machine Learning.
- **Database Technology:** PostgreSQL 15 is a mature, production-grade relational database management system with native JSON column support, excellent performance characteristics, and robust tooling. SQLAlchemy 2.0's async ORM capabilities integrate seamlessly with FastAPI's async architecture. Alembic provides version-controlled, rollback-capable schema migration management.

- **Authentication:** Firebase Authentication provides enterprise-grade, OAuth 2.0-compliant user identity management with support for Google Sign-In, email/password authentication, and JWT-based session management. The Firebase Admin SDK for Python enables server-side token verification in FastAPI without maintaining a separate authentication database.

2.1.2 Economic Feasibility

The economic feasibility analysis examines the cost-benefit relationship of the proposed system relative to available alternatives and the educational context in which it will be deployed.

- **Development Costs:** The primary development cost is human effort estimated over 12 weeks. All selected software technologies are open-source or provide free tiers adequate for academic development.
- **Infrastructure Costs:** Railway's free tier provides 500 hours of compute per month, sufficient for the academic demonstration phase. PostgreSQL is hosted on Railway's managed database service at zero cost within the free tier. Total monthly infrastructure cost for the academic project: Rs. 0.

2.1.3 Operational Feasibility

The system is designed for zero-learning-curve operation by its primary users and minimal-training operation by its secondary users. The conversational intake interface requires no instruction, users are naturally guided through the process by the AI's questions. The results page, dashboard, and admin panel follow established web application design conventions that are familiar to any smartphone user.

System maintenance is designed to be manageable by a developer with intermediate Python and JavaScript skills. Database schema changes are managed with comprehensive rollback support.

Feasibility Dimension	Rating	Key Justification
Technical	High (4/5)	Mature, well-documented stack, team competency verified;
Economic	High (5/5)	Zero development infrastructure cost;
Operational	High (4/5)	Zero-learning-curve UI; minimal maintenance burden; auto-deployment
Schedule	Medium-High (4/5)	12-week plan; AI-assisted dev with Antigravity; 2-week sprint buffer
Ethical	High (5/5)	Open-source stack; GDPR-aligned; no proprietary data; t

Table 2.1: Feasibility Analysis Summary

2.2 SOFTWARE REQUIREMENT SPECIFICATION DOCUMENT

2.2.1 Data Requirements

The system manages five categories of persistent data, each stored in a dedicated PostgreSQL table:

1. **User Identity Data (users table):** Stores the core authentication identity for each registered user. Primary attributes include a system-generated UUID primary key, the Firebase-issued unique identifier (firebase_uid) used to verify JWT tokens, the user's email address, display name, administrative privilege flag, and creation or modification timestamps.
2. **Career Profile Data (user_profiles table):** Stores the structured career profile extracted from the intake conversation. Includes all fields extracted by the skills array, interests array, personality descriptor, constraints object, career frustration text, readiness score, raw conversation history, and intake completion flag. One user may have multiple profiles with the most recent used for matching.
3. **Career Match Data (career_matches table):** Stores the three career match records associated with each completed profile. Each record includes role name, fit score, salary

range, growth outlook, growth reason, matched skills, skill gaps, time-to-readiness estimate, and the full AI-generated explanation. Three records are always created simultaneously after a matching request.

4. **Roadmap Data (roadmaps table):** Stores the personalized career development plan generated for the user's selected career match. The milestones field is a JSON array containing exactly four phase objects, each with goal, actions including resource links, measurable outcome, and checkpoint. Also stores current phase progress and completed action tracking.
5. **Feedback Check-in Data (feedback_checkins table):** Stores the record of each monthly progress check-in, including the completion rate at time of check-in, mood score, list of completed actions, stated blockers and the full AI feedback response JSON.

2.2.2 Functional Requirements

The system shall provide the following functional capabilities:

FR ID	Requirement Description	Priority	Module
FR-01	System shall initiate conversational intake with first question on page load	High	Intake

FR-02	System shall ask exactly one question per AI response turn	High	Intake
FR-03	System shall adapt questions based on information already provided	Medium	Intake
FR-04	System shall extract structured JSON profile after 6th question	High	Intake
FR-05	System shall validate profile JSON before database persistence	High	Intake
FR-06	System shall match user profile to exactly 3 ranked careers	High	Matching
FR-07	System shall provide fit score (0-100) for each career match	High	Matching
FR-08	System shall provide Indian salary range in INR for each match	High	Matching
FR-09	System shall provide growth outlook (High/Medium/Low) for each match	High	Matching
FR-10	System shall generate 4-phase roadmap for selected career	High	Roadmap
FR-11	System shall provide one free and one paid resource per action	High	Roadmap
FR-12	System shall provide measurable outcome for each phase	Medium	Roadmap

FR-13	System shall classify skill gaps as Critical/Important/Nice-to-Have	High	Gap
FR-14	System shall recommend learning resources with INR cost	High	Gap
FR-15	System shall recommend a practice project per skill gap	Medium	Gap
FR-16	System shall collect monthly check-in data (mood, actions, blockers)	Medium	Feedback

FR-17	System shall determine feedback mode based on completion rate	High	Feedback
FR-18	System shall support Google OAuth and email/password authentication	High	Auth

Table 2.2: Functional Requirements — Complete List

2.2.3 Performance Requirements

The system shall meet the following quantitative performance benchmarks, verified through load testing with Locust:

- **Frontend Performance:** Lighthouse Performance Score above 85; First Contentful Paint below 1.5 seconds on a 4G connection; Time to Interactive below 3.5 seconds; Cumulative Layout Shift below 0.1.
- **Database Query Performance:** All ORM-generated queries shall execute within 100 milliseconds on the production database with indices maintained on `firebase_uid`, `user_id` foreign keys, `profile_id` foreign keys, and `career_match_id` foreign keys.
- **Concurrent User Capacity:** The system shall support 500 concurrent non-AI requests and 50 concurrent AI requests without error rates exceeding 1%, leveraging Railway's horizontal scaling capabilities for backend expansion and Vercel's global CDN for frontend distribution.

2.2.4 Dependability Requirements

The system shall meet the following reliability and availability standards:

- **System Uptime:** The platform shall maintain 99.5% monthly uptime (maximum 3.65 hours downtime per month).
- **Data Durability:** PostgreSQL data shall be protected by Railway's automated daily backups with a 7-day retention window. SQLAlchemy's connection pool health checking ensures stale connections are replaced before use.
- **Error Recovery:** All API endpoints implement comprehensive error handling. Application-level exceptions are caught, logged, and translated to appropriate HTTP responses.
- **Graceful Degradation:** Previously generated data (profiles, matches, roadmaps) shall remain accessible from the database throughout any AI service interruption.

2.2.5 Maintainability Requirements

The system is designed for long-term maintainability by developers with intermediate Python and JavaScript competency:

- **Code Organisation:** All AI system prompts are defined as named string separated from business logic and API routing. A developer can modify any AI prompt without touching any other code file.
- **Database Evolution:** All schema changes are managed through Alembic revision scripts stored in the alembic/versions/ directory. The migration history provides a complete audit trail of all schema changes.
- **Documentation:** All Python functions include Google-style docstrings with parameter types, return types, and exception descriptions. The FastAPI framework automatically generates and maintains an OpenAPI specification accessible at docs that documents all endpoints, request or response schemas and authentication requirements.
- **Dependency Management:** Python dependencies are pinned to specific versions in requirements.txt. JavaScript dependencies are locked in package-lock.json. Dependabot alerts are configured on the GitHub repository for security vulnerability notifications.
- **Configuration Management:** All environment-specific configuration values such as API keys, database URLs, CORS origins, Firebase credentials are managed through .env files. A documented .env.example file lists all required variables with descriptions but no values by enabling any developer to configure a new environment without consulting existing team members.

2.2.6 Security Requirements

The system implements a multi-layered security architecture:

Security Control	Implementation	Standard
Authentication	Firebase JWT verified on every protected request	OAuth 2.0 / OIDC
Authorisation	Role-based access; admin endpoints require is_admin=true	RBAC
Data Isolation	All queries filtered by authenticated user ID	Principle of Least Privilege

Transport Security	HTTPS enforced on all endpoints via Vercel/Railway TLS	TLS 1.3
Secret Management	All keys in environment variables; .env never committed	12-Factor App
Input Validation	Pydantic models validate all request bodies server-side	Input Sanitisation
CORS Policy	Restricted to approved frontend domain only	CORS Standard
API Rate Limiting	Railway platform rate limiting; API has built-in limits	Rate Limiting
SQL Injection Prevention	SQLAlchemy ORM parameterised queries used exclusively	OWASP Top 10

Dependency Security	Dependabot alerts; regular dependency updates	OWASP Top 10
---------------------	---	--------------

Table 2.3: Security Requirements and Implementation

2.2.7 Look and Feel Requirements

The visual design and interaction model of the system shall conform to the following requirements:

- **Design System:** All UI components shall be implemented using React functional components with Tailwind CSS utility classes.
- **Typography:** Body text shall use the system sans-serif font stack for optimal rendering across operating systems. Minimum body text size shall be 16px to meet requirements.
- **Responsive Layout:** The application shall use Tailwind's responsive breakpoint system (sm:, md:, lg:, xl:) to ensure that all layouts are fully functional at screen widths from 375px to 2560px. The intake chat interface shall occupy the full viewport on all screen sizes.
- **Interaction Design:** All interactive elements shall provide visual feedback on hover (colour change, shadow addition) and on click (scale reduction, colour change). Loading states shall be indicated by animated spinner components with descriptive loading messages. Error states shall be indicated by red-bordered input fields with

descriptive error text below the field.

- **Accessibility:** All images shall include descriptive alt text. All interactive elements shall be keyboard-navigable. Focus indicators shall be visible and meet WCAG 2.1 AA contrast requirements. ARIA labels shall be applied to all custom interactive components (chat input, progress indicators, career cards).

2.3 VALIDATION

The following validation rules are enforced at the API layer using Pydantic models, with corresponding client-side validation in React:

2.3.1 Intake Validation

The intake session start endpoint (POST /api/intake/start) requires no request body. The intake respond endpoint (POST /api/intake/respond) validates that session_id is a non-empty string, message is a non-empty string of maximum 2,000 characters, and history is a list of objects each containing a role field ('user' or 'assistant') and a content string field.

2.3.2 Career Match Validation

The career match endpoint (POST /api/careers/match) validates the profile object for presence of required fields (education, skills, interests, personality, constraints, frustration, readiness_score), correct data types (skills and interests as arrays, readiness_score as integer 0-10, constraints as object with string values), and minimum content (skills array must contain at least one entry, education string must be non-empty).

2.4 EXPECTED HURDLES

The following technical and operational challenges were anticipated prior to implementation and addressed through specific mitigation strategies:

2.4.1 AI Output Consistency (High Risk)

Large language models, despite being highly capable, do not guarantee perfectly consistent output format on every invocation. Early testing revealed occasional responses where produced JSON outside the expected XML tags, or where array fields contained fewer items

than specified. Mitigation strategy implemented: (a) XML tag delimiters with explicit instructions to output 'ONLY' the specified JSON structure; (b) retry logic with maximum three attempts before returning an error to the user; (c) detailed logging of raw responses for debugging; (d) specific prompt examples demonstrating the exact expected format.

2.4.2 Firebase Configuration Across Environments (Medium Risk)

Firebase Authentication requires separate project configuration for development and production environments, with distinct API keys, OAuth client IDs, and authorised domains. Failure to properly configure environment separation can result in authentication tokens from the development environment being accepted in production, or vice versa. Mitigation strategy implemented: separate Firebase projects for development ('career-counsellor-dev') and production ('career-counsellor-prod'), with all configuration values managed through environment variables and documented in the .env.example template.

2.4.3 PostgreSQL JSON Query Performance (Low Risk)

Querying within JSON column values (skills, interests, milestones) using PostgreSQL's JSON operators can be significantly slower than querying standard indexed columns, particularly as the dataset grows. Mitigation strategy implemented: (a) GIN (Generalised Inverted Index) indices on the skills and interests JSON columns in user_profiles; (b) application-level caching of frequently accessed career matches; (c) query analysis using EXPLAIN ANALYZE for all complex queries.

2.5 SDLC MODEL TO BE USED

This project adopts the Agile Iterative Development model, structured as six two-week sprints with defined deliverables at the end of each sprint. This model was selected over the alternatives (Waterfall, Spiral, RAD) for the following specific reasons:

2.5.1 Rationale for Agile Selection

AI Prompt Engineering Uncertainty: The behaviour of large language models in response to specific system prompts cannot be fully predicted theoretically it must be empirically observed and iteratively refined. The Agile model's emphasis on short feedback cycles and willingness to revise designs based on observed behaviour is uniquely well-suited to this requirement. Waterfall's sequential, specification-first approach would have committed the project to

specific prompt designs before their effectiveness could be empirically validated.

Modular Architecture Alignment: The six AI modules of the system (intake, matching, roadmap, gap, feedback, admin) each represent a self-contained functional unit with clear inputs and outputs. This modularity maps naturally to six independent Agile sprints, with each sprint delivering a working, testable module that can be demonstrated before the next sprint begins.

Continuous Verification with Antigravity: Google Antigravity's browser-based verification capability which enables AI agents to autonomously test newly built features in a live browser environment aligns perfectly with the Agile principle that working software is the primary measure of progress. Every sprint ends with a browser demonstration of the newly built functionality.

2.5.2 Sprint Plan

Sprint	Duration	Deliverables	Acceptance Criteria
Sprint 1	Weeks 1-2	Project setup, DB models, Firebase auth, CI/CD	All 5 DB tables created; login with Google works; /health returns 200
Sprint 2	Weeks 3-4	intake API, intake chat UI	Full 6-question intake completes and extracts profile JSON

Sprint 3	Weeks 5-6	Career matching API, results page, career selection	3 career cards display with fit scores after intake completion
Sprint 4	Weeks 7-8	Roadmap generator, gap analysis, results integration	Selecting a career generates roadmap timeline and gap table
Sprint 5	Weeks 9-10	Feedback loop, user dashboard, check-in flow	Monthly check-in submits and displays AI feedback in correct mode
Sprint 6	Weeks 11-12	Admin panel, comprehensive testing, deployment, docs	All test cases pass;

CHAPTER-3

SYSTEM DESIGN

SYSTEM DESIGN

3.1 DESIGN APPROACH

The system employs an Object-Oriented Design (OOD) approach throughout all layers of the application, with functional-oriented design applied at the AI service level where pure functions with defined inputs and outputs are more appropriate.

3.1.1 Backend Architecture

The FastAPI backend is organized into four distinct layers, each with well-defined responsibilities and clear inter-layer contracts:

- **Routing Layer (`app/api/routes/`):** Contains the FastAPI route handler functions for all six endpoint groups. Route handlers are responsible only for request parsing, authentication verification, and response formatting. Business logic is delegated to the service layer.
- **Service Layer (`app/services/`):** Contains the core business logic and AI interaction code. The class encapsulates all Anthropic API calls, with one method per AI function. The `CareerMatcherService`, `RoadmapGeneratorService`, and `GapAnalyserService` classes process AI outputs and prepare data for persistence.
- **Data Access Layer (`app/models/` and `app/schemas/`):** SQLAlchemy ORM models define the database schema and provide type-safe database interaction. Pydantic schemas define the request and response contracts for all API endpoints, providing automatic validation and serialisation.
- **Core Configuration Layer (`app/core/`):** Centralised configuration (Settings class with environment variable loading), database connection management (async SQLAlchemy engine and session factory), and security utilities (Firebase token verification function).

3.1.2 Frontend Architecture

The React frontend follows a unidirectional data flow architecture using Zustand for state management:

- **Page Components (src/pages/):** Top-level components that correspond to URL routes. Pages compose multiple functional components and coordinate data fetching through custom hooks.
- **Functional Components (src/components/):** Reusable, single-responsibility UI components. Components receive data as props and are fully controlled (no internal state beyond UI-specific concerns like open/closed modal state).
- **Custom Hooks (src/hooks/):** React hooks that encapsulate complex data fetching and state update logic (useIntakeFlow, useCareerMatch, useAuth). Hooks interface between components and Zustand stores.
- **Zustand Stores (src/store/):** Three independent state stores (userStore, intakeStore, careerStore) manage their respective domains. Stores are persisted to localStorage where appropriate.
- **API Layer (src/api/):** Axios-based API client functions organised by domain. All API calls are encapsulated in dedicated files (intakeApi.js, careerApi.js, userApi.js) with consistent error handling.

3.2 SYSTEM DESIGN USING STRUCTURED ANALYSIS AND DESIGN TOOLS

3.2.1 Data Flow Diagrams

Data Flow Diagrams (DFDs) are used to represent the flow of data through the system at three levels of abstraction:

- **Level 0 DFD (Context Diagram):** The context diagram represents the AI Career Counsellor as a single process (Process 0: AI Career Counsellor System) interacting with three external entities. The Student/Job Seeker entity sends intake responses, career selections, and progress check-in data to the system, and receives career matches, roadmap milestones, and feedback reports.

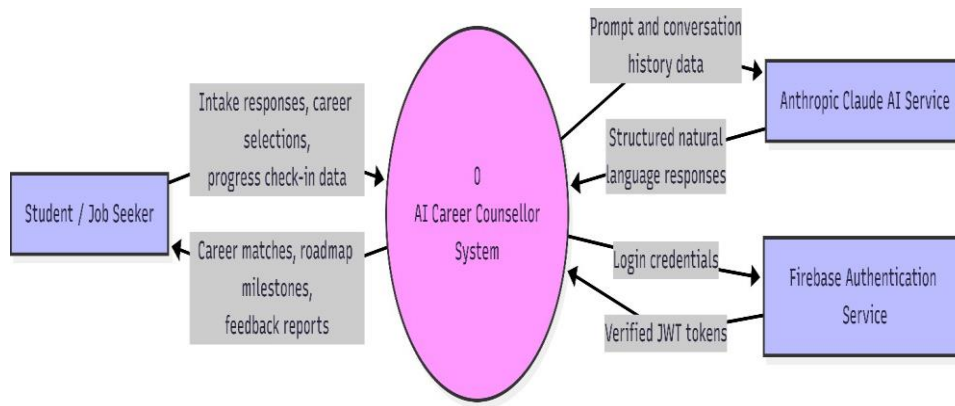


Figure 3.1: Level 0 DFD — Context Diagram

- Level 1 DFD:** At Level 1, the system decomposes into six major processes, five data stores, and all external data flows: Process 1.0 Authenticate User (interfaces with Firebase, produces verified session); Process 2.0 Conduct Career Intake, Process 3.0 Match Career Paths (reads from D2, writes to D3); Process 4.0 Generate Career Roadmap (reads from D3, writes to D4); Process 5.0 — Analyze Skill Gaps (reads from D3, writes to D5); Process 6.0 Process Feedback Check- in (reads from D4 and D5, writes to D6). Data stores: D1: User Records, D2: Career Profiles, D3: Career Matches, D4: Roadmaps, D5: Gap Analyses, D6:

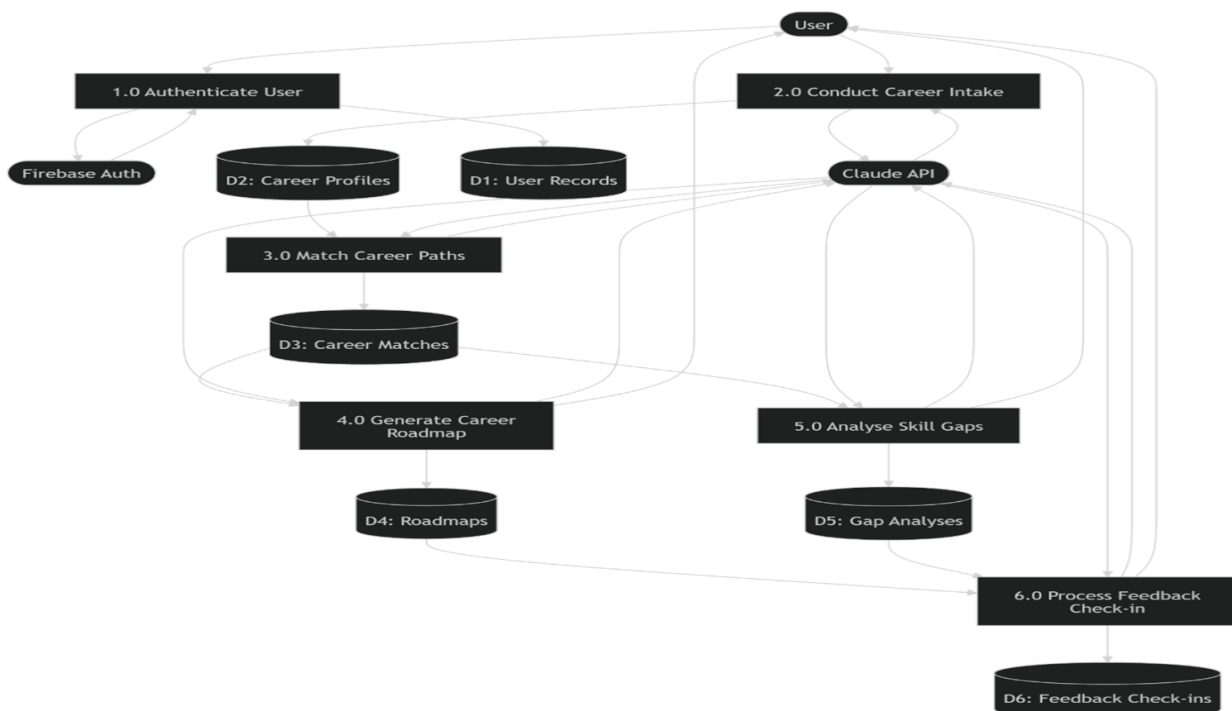


Figure 3.2: Level 1 DFD — System Decomposition into Six Processes

- Level 2 DFD (AI Processing Module):** At Level 2, Process 3.0 (Match Career Paths) decomposes into: Process 3.1 Format Profile for Prompt , Process 3.2 sends formatted prompt, receives raw text response; Process 3.3 Extract JSON from Response (parses XML tags, extracts JSON block); Process 3.4 Validate Match Schema (checks for required fields, fit score range, salary range validity); Process 3.5 Persist Match Records (saves three match records to D3). A feedback loop from Process 3.4 to Process 3.2 represents the retry mechanism for invalid responses.

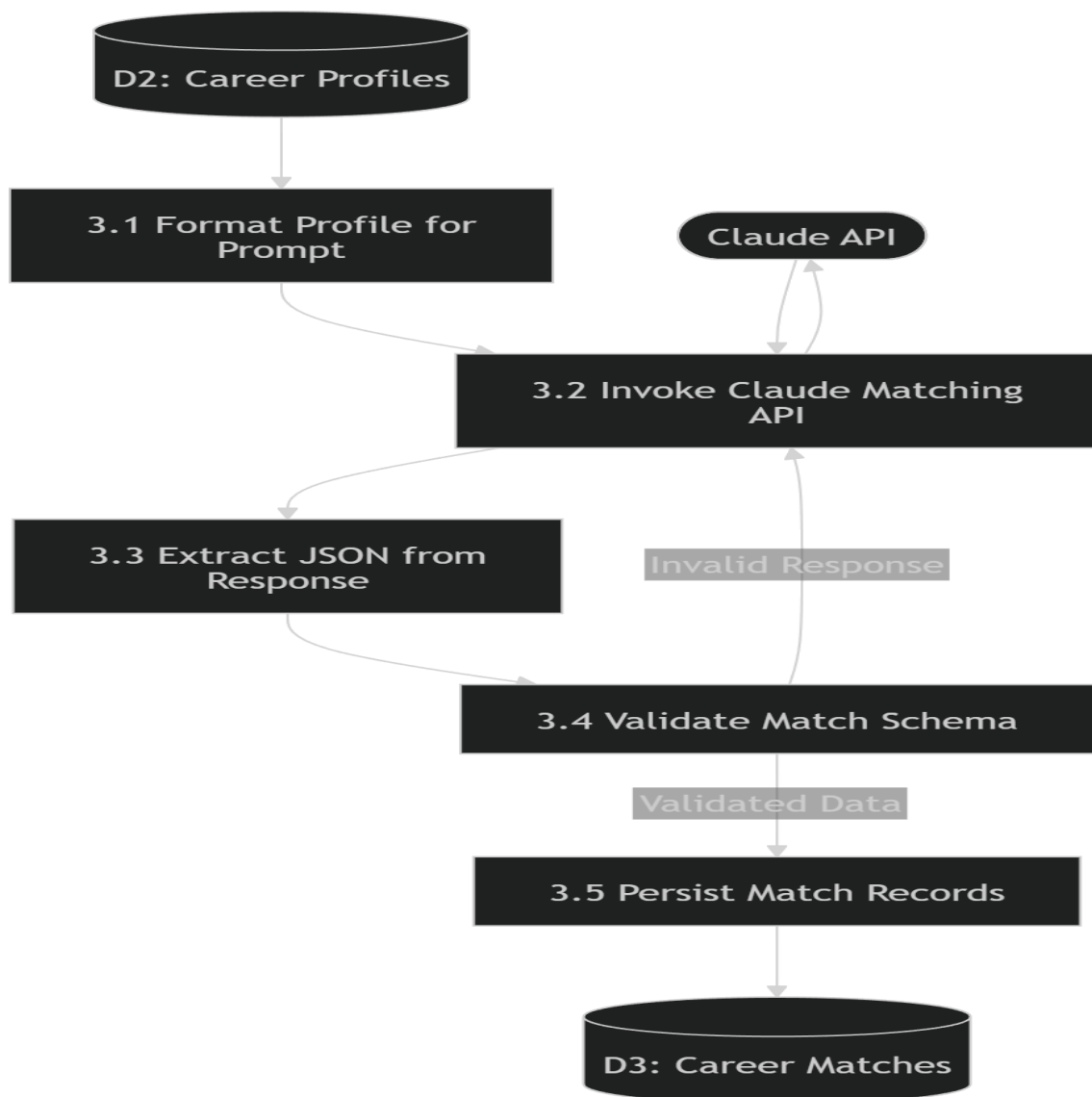


Figure 3.3: Level 2 DFD — Career Matching Sub-process Decomposition

3.2.2 Data Dictionary

The following comprehensive data dictionary defines all primary data elements flowing through the system:

Element	Type	Length/Range	Description	Validation Rule
firebase_uid	String	28 chars	Firestore user unique ID	Non-empty, unique across users table
email	String	Max 254 chars	User email address	Valid email format (RFC 5322)
skills	JSON Array	1–20 items	List of user skill strings	Min 1, max 20 items; each max 100 chars
interests	JSON Array	1–15 items	List of user interest strings	Min 1, max 15 items
readiness_score	Integer	0–10	AI career readiness level	Integer, inclusive range 0–10
fit_score	Float	0.0–100.0	Career match compatibility	Float, inclusive range 0–100
salary_min_inr	Integer	100000–99999999	Min annual salary (INR)	Positive integer, less than salary_max
salary_max_inr	Integer	100000–99999999	Max annual salary (INR)	Positive integer, greater than salary_min
growth_outlook	Enum String	High/Med/Low	Career demand growth classification	Must be one of: High, Medium, Low
priority	Enum String	3 options	Skill gap urgency level	Must be: CRITICAL, IMPORTANT, NICE-TO-HAVE

milestones	JSON Array	Exactly 4 items	Career roadmap phases	Must contain exactly 4 phase objects
completion_rate	Float	0.0–100.0	Roadmap progress percentage	Float, inclusive range 0–100
mood_score	Integer	1–10	User self-reported motivation	Integer, inclusive range 1–10
weekly_hours	Integer	1–40	Weekly study time commitment	Positive integer, max 40 hours
session_id	String/UUID	36 chars	Intake session identifier	Valid UUID v4 format

Table 3.1: Data Dictionary — Core Data Elements

3.2.3 Structured Charts

The system structure chart represents the hierarchical decomposition of the AI Career Counsellor into its constituent modules and sub-modules:

Level 0 — System: AI Career Counsellor. Level 1 — Major Subsystems: (1) Authentication Subsystem, (2) Intake Subsystem, (3) Matching Subsystem, (4) Development Planning Subsystem, (5) Progress Tracking Subsystem, (6) Administration Subsystem. Level 2 — Modules: Authentication decomposes into Firebase Token Verifier and User Registration Handler. Intake decomposes into Session Manager and Profile Extractor.

3.2.4 UML Diagrams

The following UML diagrams provide precise, standard-notation representations of the system's structural and behavioural design:

Use Case Diagram: The system has two primary actors — Student/Job Seeker and Admin/Faculty Counsellor. The Student actor is associated with nine use cases: Register/Login (extends to Google OAuth Login and Email Login); Complete Career Intake (includes Start Session, Respond to Questions, and Review Profile); View Career Matches (includes Select Target Career); View Career Roadmap (includes Mark Actions Complete, Expand Phase Details); View Skill Gap Analysis (includes Filter by Priority, Access Resource Links); Update

Progress (includes Log Completed Actions); Submit Monthly Check-in (includes Rate Mood, Report Blockers); View AI Feedback; and Update Career Target (alternative flow). The Admin actor is associated with four use cases: View All Users (includes Search/Filter Users, Sort by Status); View User Profile Details (includes View Career Match, View Roadmap); View Cohort Statistics; and Export Reports as CSV.

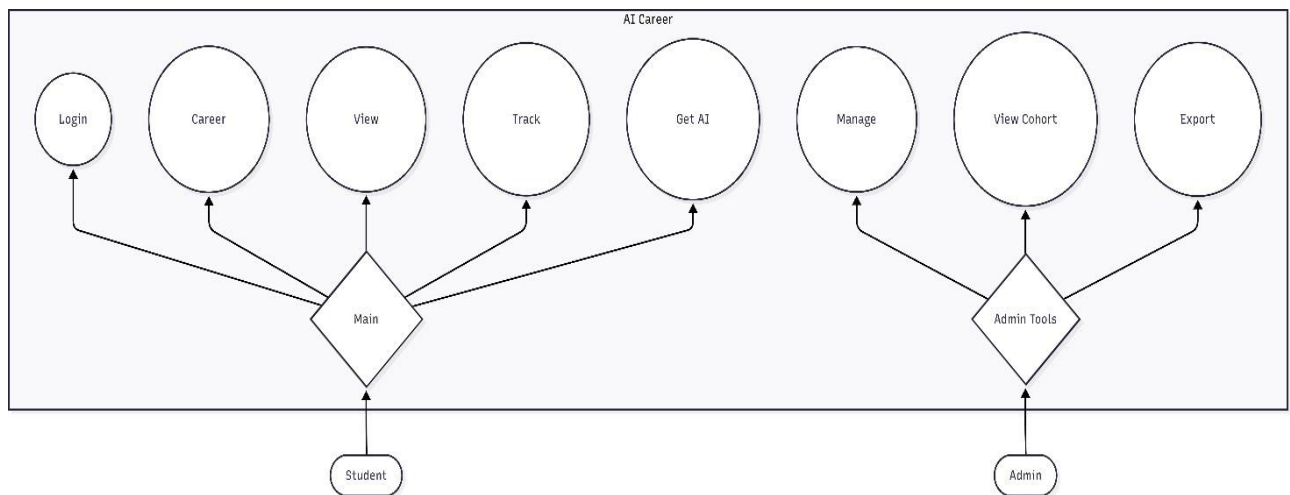


Figure 3.4: Use Case Diagram — Student and Admin Actors

Sequence Diagram — Intake Flow: The sequence diagram for the intake engine flow illustrates eight participant lifelines: Browser (Student), React IntakePage Component, intakeStore (Zustand), Axios API Client, FastAPI Intake Router, and PostgreSQL Database. The sequence begins with the student opening the intake page. IntakePage calls `startIntake()` in `intakeApi.js`. Axios sends `POST /api/intake/start` to FastAPI Router. Router returns JSON response. Axios resolves the promise. IntakePage dispatches the AI message to intakeStore via `addMessage('assistant', question)`. IntakePage renders the chat bubble. The student types their answer and presses send. IntakePage calls `respondToIntake(sessionId, userMessage, history)`. This loop repeats for questions 2 through 6. After question 6, it detects the profile XML tag and calls `extract_json_from_tag()`, and returns `{is_complete: true, profile: {...}}`. IntakePage dispatches `setProfile(profile)` to intakeStore. Router saves profile to PostgreSQL via UserProfile model. IntakePage redirects to `/results` with the user's `profile_id`.

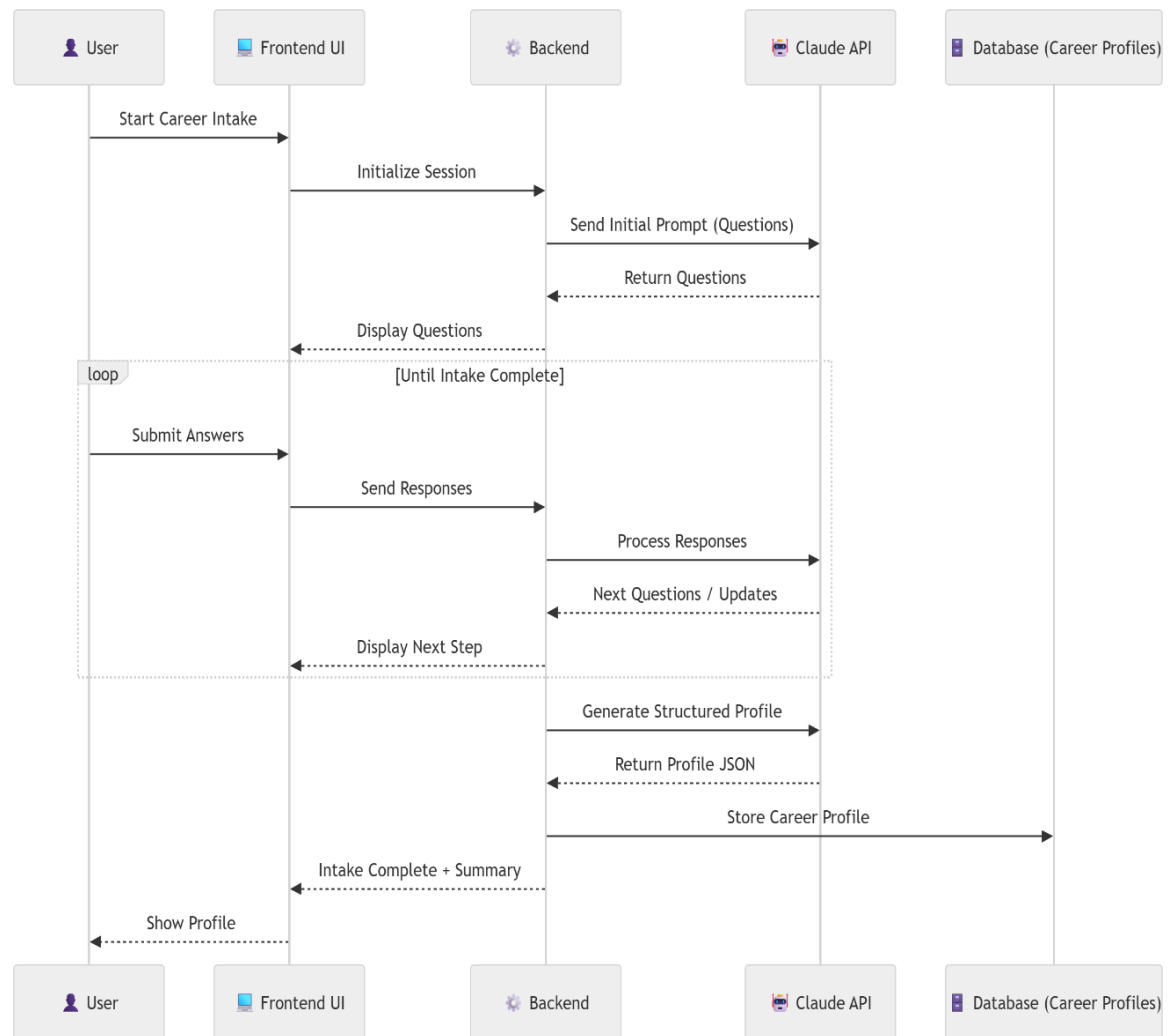


Figure 3.5: Sequence Diagram — Complete Intake Engine Flow

Class Diagram (Backend Service Layer): The backend class diagram shows five primary service classes. It has private attributes `client` (Anthropic client instance), `INTAKE_PROMPT` (string constant), `MATCH_PROMPT` (string constant), `ROADMAP_PROMPT` (string constant), `GAP_PROMPT` (string constant), `FEEDBACK_PROMPT` (string constant), and public methods `run_intake_turn(history: List, message: str) -> Dict`, `match_careers(profile: Dict) -> List`, `generate_roadmap(profile: Dict, career: Dict) -> Dict`, `analyse_gaps(profile: Dict, career: Dict) -> Dict`, `run_feedback(roadmap: Dict, completed: List, mood: int, blockers: List) -> Dict`, and private method `extract_json_from_tag(text: str, tag: str) -> Dict`. `UserProfileService` depends on `Service` and has methods `create_profile(user_id: UUID, answers: List) -> UserProfile` and `get_latest_profile(user_id: UUID) -> UserProfile`. `CareerMatchService` has methods `generate_matches(profile: UserProfile) -> List[CareerMatch]` and `get_matches(profile_id:`

UUID) -> List[CareerMatch]. RoadmapService.

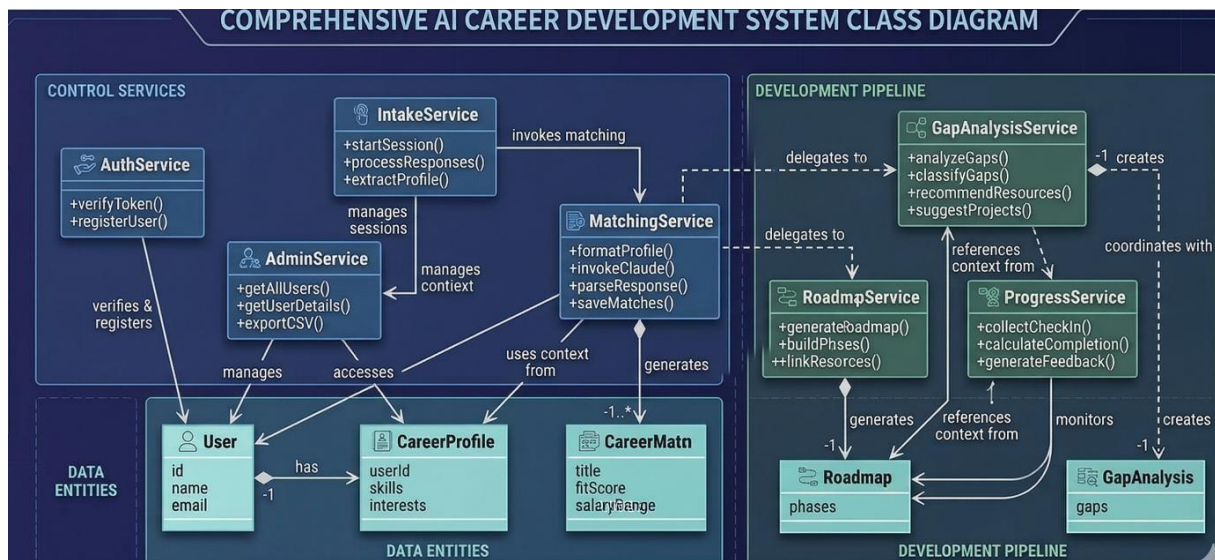


Figure 3.6: Class Diagram — Backend Service Layer

Activity Diagram: The activity diagram for the end-to-end user flow begins with the initial node at the Landing Page. Decision node 1: Is user authenticated? If No, proceed to Login/Register Page (Google OAuth or email). If Yes, decision node 2: Has user completed intake? If No, proceed to Intake Page conduct six-question conversation extract profile save to database. If Yes, skip to results. After intake, proceed to Match Careers invoke matching API display three career match cards. Decision node 3: Has user selected a career? If No, wait. If Yes, parallel fork generate roadmap AND analyse gaps concurrently. Both join at the Results Page display roadmap timeline and gap table. Proceed to Dashboard track progress mark actions complete. Timer-based decision node: 30 days elapsed? If Yes, proceed to Check-In Modal collect mood, completed actions, blockers calculate completion rate. Decision node 4: Completion rate? If below 50%, REPLAN mode generate revised plan. If 50-80%, NUDGE mode generate targeted encouragement. If above 80%, ACCELERATE mode generate stretch goal. All three modes converge at Display AI Feedback. Return to Dashboard. The activity diagram includes a 'Change Target Career' alternative flow from the Dashboard back to the career selection step.



Figure 3.7: Activity Diagram — Complete End-to-End User Journey

3.3 USER INTERFACE DESIGN

The user interface is designed following the principles of progressive disclosure by showing only what is needed at each step, conversational familiarity, and information density management.

3.3.1 Landing Page Design

The landing page is structured in six sections from top to bottom: (1) Navigation bar with logo, sign-in button, and 'Start Free' CTA; (2) Hero section with headline ('Find the career built for you'), 150-word value proposition, animated career title rotation (Data Scientist / Cloud Engineer / Product Manager / AI Engineer cycling every 2 seconds), and two CTA buttons ('Start Free Assessment' primary, 'Watch Demo' secondary); (3) Feature grid — three cards with icons for 'AI Career Matching', 'Personalised Roadmap', and 'Skill Gap Analysis'; (4) How It Works — four numbered steps with connecting arrows (Complete Intake → Get Career Matches → Build Roadmap → Track Progress); (5) Testimonials — three student quote cards

with name, institution, and career outcome; (6) Footer with social links, legal links, and college affiliation.

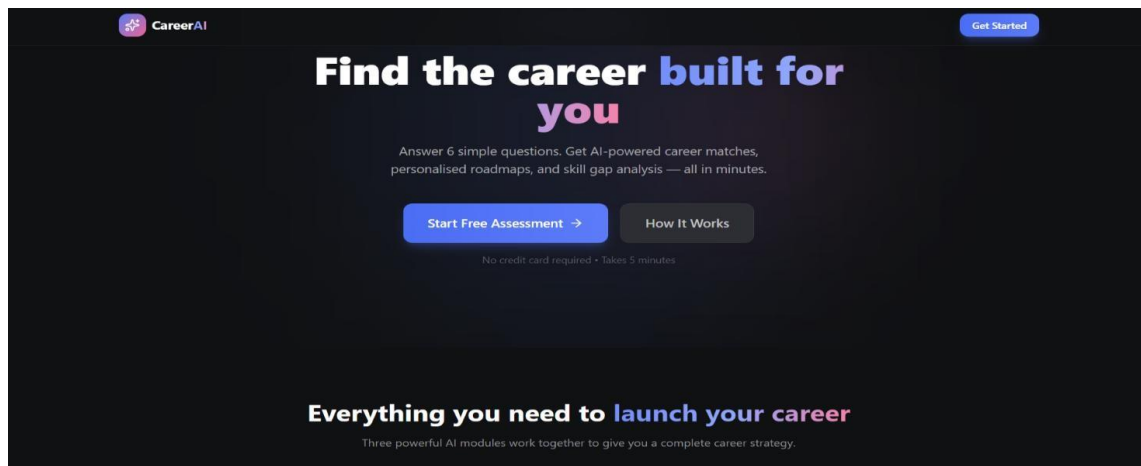


Figure 3.8: Landing Page Wireframe — Six-Section Layout

3.3.2 Intake Chat Interface Design

The intake page uses a full-height chat layout with three zones: (1) Progress bar at the top showing current question number and proportional fill (e.g., 3/6 = 50% filled); (2) Chat message area in the middle — scrollable, with AI messages as left-aligned light-grey bubbles with AI avatar initials, and user messages as right-aligned primary-colour bubbles. A typing animation (three pulsing dots) appears in a grey bubble while awaiting AI response; (3) Input zone at the bottom — a full-width text input field with placeholder 'Type your answer here...', send button (arrow icon), and Enter-key submission support. On mobile, the keyboard pushes the input zone up, keeping it visible.

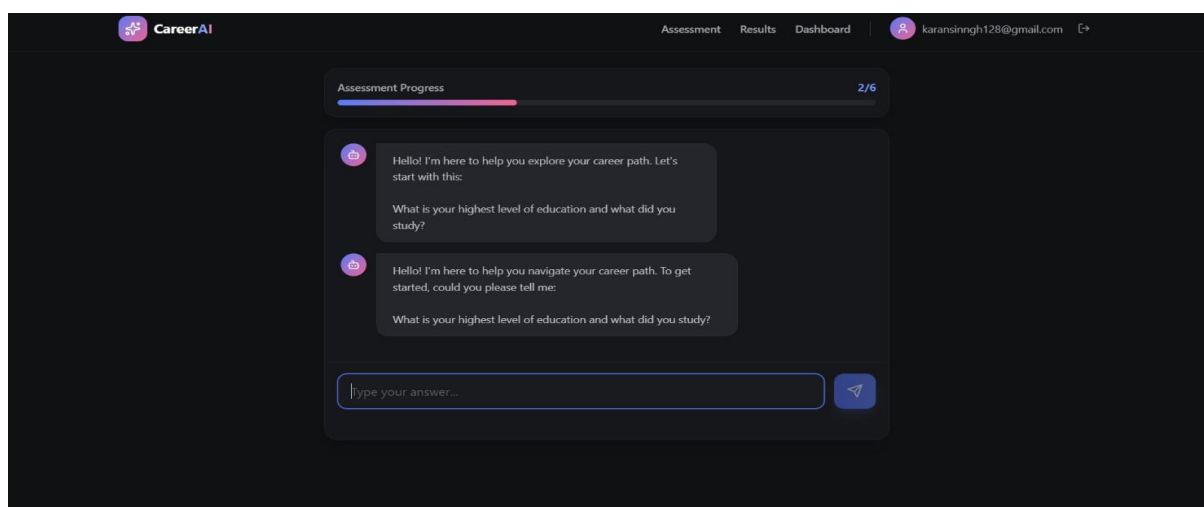


Figure 3.9: Intake Chat Interface Wireframe

3.3.3 Results Page Design

The results page is structured in three vertical sections: Section 1 — Career Matches (full width): Three cards in a responsive grid (1 column mobile, 3 columns desktop). Each card contains: role title (large, bold), fit score as a circular progress indicator (0-100 with colour coding — green for 80+, amber for 60-79, red for below 60), INR salary range badge, growth outlook tag (High/Medium/Low with colour), and top 3 required skills as pills. Active/selected card has a distinct border and 'Selected' badge. Section 2 — Roadmap Timeline (two-thirds width): A vertical timeline with four phase nodes. Each node shows phase name, timeframe, and status icon. Clicking a node expands to show goal statement, action list with resource links and completion checkboxes, measurable outcome, and checkpoint text. Section 3 — Gap Analysis Table (one-third width): A table with priority filter tabs above. Table columns: Skill, Priority Badge, Hours, Free Resource (link), Paid Resource (link + cost), and Practice Project.

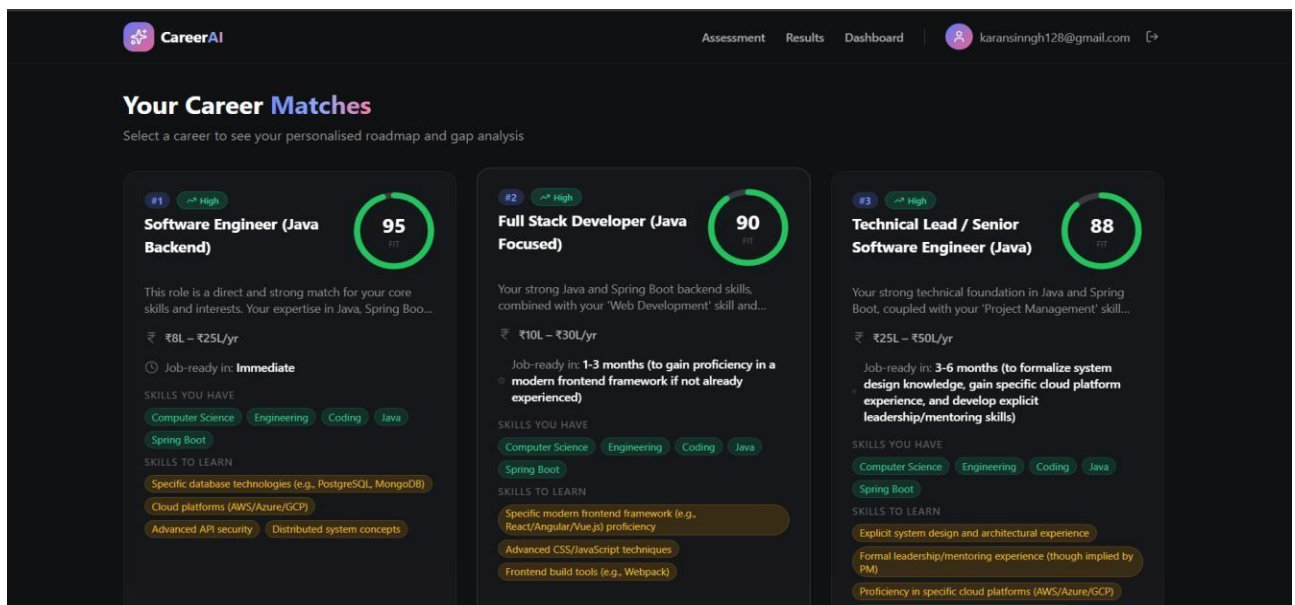


Figure 3.10: Results Page Wireframe — Three-Section Layout

3.3.5 Dashboard Design

The dashboard is designed for quick progress orientation and periodic check-in initiation. Layout: (1) Summary card spanning full width — circular phase completion gauge (current phase actions completed/total), phase name, next immediate action highlighted, and overall roadmap timeline position indicator; (2) Three metric cards in a row — 'Current Phase', 'Completion Rate %', and 'Days to Next Milestone'; (3) Weekly commitment tracker — planned vs. logged hours for current week as a horizontal bar chart; (4) 'Submit Monthly Check-in' button (prominent, centred); (5) Condensed roadmap timeline (collapsed view, expandable);

(6) Recent feedback card — showing the most recent AI feedback response with mode badge.

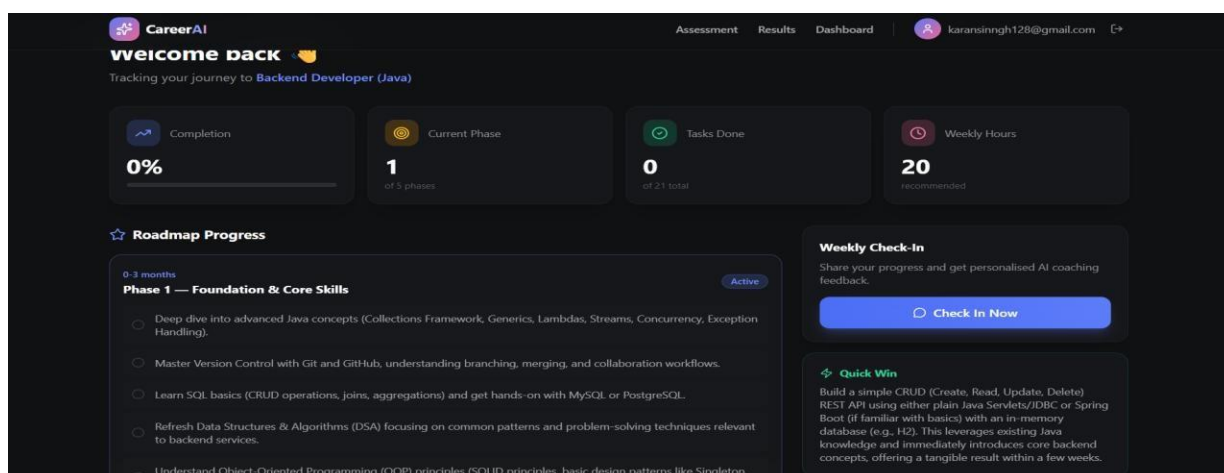


Figure 3.11: Dashboard Page Wireframe

3.4 DATABASE DESIGN

The database is implemented in PostgreSQL 15, managed through SQLAlchemy 2.0 ORM models and Alembic migration scripts. All tables use UUID primary keys generated server-side to avoid sequential ID enumeration vulnerabilities.

3.4.5 Users Table

Column Name	Data Type	Constraints	Default	Description
id	UUID	PK, NOT NULL	uuid_generate_v4()	System-generated primary key
firebase_uid	VARCHAR(128)	UNIQUE, NOT NULL, INDEX	—	Firebase authentication UID
email	VARCHAR(254)	UNIQUE, NOT NULL	—	User email address
full_name	VARCHAR(200)	NULLABLE	NULL	Display name from Firebase profile
is_admin	BOOLEAN	NOT NULL	FALSE	Administrative privilege flag
created_at	TIMESTAMP WITH TZ	NOT NULL	NOW()	Record creation timestamp

Table 3.2: users Table — Full Schema Definition

3.4.6 User Profiles Table

Column Name	Data Type	Constraints	Default	Description
id	UUID	PK, NOT NULL	uuid_generate_v4()	System-generated primary key
user_id	UUID	FK(users.id), NOT NULL, INDEX	—	Reference to owning user
education	VARCHAR(500)	NULLABLE	NULL	Educational background text

skills	JSONB	NOT NULL	[]	Array of skill strings (GIN indexed)
interests	JSONB	NOT NULL	[]	Array of interest strings (GIN indexed)
personality	VARCHAR(200)	NULLABLE	NULL	AI-extracted personality descriptor
constraints	JSONB	NOT NULL	{}	Location, salary_min, timeline object
frustration	TEXT	NULLABLE	NULL	User's primary career frustration
readiness_score	SMALLINT	NOT NULL, CHECK(0-10)	0	AI-assessed career readiness
raw_answers	JSONB	NOT NULL	[]	Full intake conversation history
intake_completed	BOOLEAN	NOT NULL	FALSE	Whether intake was completed
created_at	TIMESTAMP WITH TZ	NOT NULL	NOW()	Profile creation timestamp

Table 3.3: user_profiles Table — Full Schema Definition

3.4.7 Career Matches Table

Column Name	Data Type	Constraints	Description
id	UUID	PK, NOT NULL	System-generated primary key
profile_id	UUID	FK(user_profiles.id), NOT NULL, INDEX	Reference to source profile
rank	SMALLINT	NOT NULL, CHECK(1-3)	Match rank (1 = best fit)

role	VARCHAR(200)	NOT NULL	Career role title
fit_score	NUMERIC(5,2)	NOT NULL, CHECK(0-100)	Compatibility score 0-100
why_matched	TEXT	NOT NULL	AI explanation of match rationale
salary_min_inr	INTEGER	NOT NULL, CHECK(>0)	Minimum annual salary in INR
salary_max_inr	INTEGER	NOT NULL, CHECK(>0)	Maximum annual salary in INR
growth_outlook	VARCHAR(10)	NOT NULL	High/Medium/Low growth classification
growth_reason	TEXT	NOT NULL	Explanation of growth outlook
required_skills	JSONB	NOT NULL, DEFAULT []	All skills required for role
user_has	JSONB	NOT NULL, DEFAULT []	User's matching skills
user_lacks	JSONB	NOT NULL, DEFAULT []	Skills user still needs
time_to_job_ready	VARCHAR(100)	NOT NULL	Estimated time to employability

created_at	TIMESTAMP WITH TZ	NOT NULL	Record creation timestamp
------------	----------------------	----------	---------------------------

Table 3.4: career_matches Table — Full Schema Definition

3.5 ER Diagrams

The Entity-Relationship (ER) diagram for the AI Career Counsellor database schema shows five entities connected through four relationships:

Entity 1 — USERS: Attributes: id (PK), firebase_uid, email, full_name, is_admin, created_at, updated_at. Entity 2 — USER_PROFILES: Attributes: id (PK), user_id (FK), education, skills, interests, personality, constraints, frustration, readiness_score, raw_answers, intake_completed, created_at. Entity 3 — CAREER_MATCHES: Attributes: id (PK), profile_id (FK), rank, role, fit_score, why_matched, salary_min_inr, salary_max_inr, growth_outlook, growth_reason, required_skills, user_has, user_lacks, time_to_job_ready, created_at. Entity 4 — ROADMAPS: Attributes: id (PK), career_match_id (FK), target_role, total_duration, milestones, weekly_commitment_hours, quick_win, biggest_risk, current_phase, created_at. Entity 5 — FEEDBACK_CHECKINS: Attributes: id (PK), roadmap_id (FK), check_in_date, completion_rate, mood_score, completed_actions, blockers, mode, ai_response.

Relationships: USERS (1) → USER_PROFILES (M): One user can create multiple profiles (if they restart the intake). Cardinality 1:N. USER_PROFILES (1) → CAREER_MATCHES (3): Each profile generates exactly three career match records. Cardinality 1:3. CAREER_MATCHES (1) → ROADMAPS (0 or 1): A roadmap is generated only for the career match the user selects. Not all three matches will have roadmaps. Cardinality 1:0..1. ROADMAPS (1) → FEEDBACK_CHECKINS (M): A roadmap accumulates multiple check-in records over time. Cardinality 1:N.

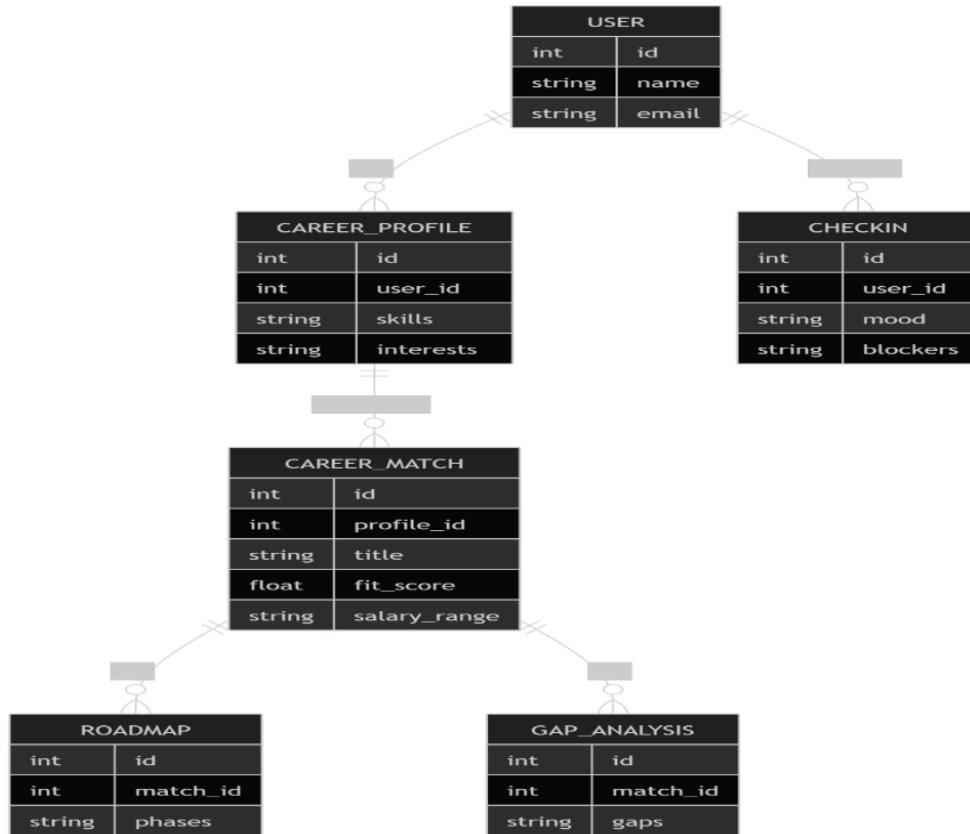


Figure 3.12: Entity-Relationship Diagram — Full Database Schema

3.6 NORMALISATION

The database schema is designed to Third Normal Form (3NF), with the following analysis by table:

Table	1NF	2NF	3NF	Notes
users	Yes — atomic values, single PK	Yes — no composite PK	Yes — no transitive deps	firebase_uid → email is not transitive; both depend directly on id
user_profiles	Yes — JSONB for arrays is atomic at column level	Yes — all attrs depend on id	Yes — no transitive deps	skills and interests are logically multi-valued but treated as atomic JSON columns
career_matches	Yes — all columns atomic	Yes — all attrs depend on id	Yes — no transitive deps	salary_min and salary_max depend on id, not on role (role is not a key)

roadmaps	Yes — milestones as JSONB	Yes — all attrs depend on id	Yes — no transitive deps	current_phase depends directly on roadmap_id, not on career_match_id
feedback_checkins	Yes — all columns atomic	Yes — all attrs depend on id	Yes — no transitive deps	mode is determined by completion_rate but stored for performance/auditability

Table 3.5: Normalisation Analysis — All Tables

3.7 DATABASE MANIPULATION

All database operations are performed through SQLAlchemy 2.0's async ORM interface using the repository pattern. An AsyncSession is injected into each route handler through FastAPI's dependency injection system (Depends(get_db)). The session factory is configured with pool_size=10, max_overflow=20, pool_pre_ping=True (health check on connection checkout), and pool_recycle=3600 (recycle connections after one hour to prevent staleness).

Write Operations: All INSERT and UPDATE operations are wrapped in SQLAlchemy async context managers that automatically commit on success and rollback on any exception. Bulk inserts (creating all three career match records simultaneously after a matching operation) use db.add_all([match1, match2, match3]) followed by a single await db.commit() for atomicity.

Read Operations: The selectinload() strategy is used for all relationship traversals (e.g., loading a user's profiles with their associated career matches) to prevent the N+1 query problem. All list queries include explicit LIMIT clauses to prevent unbounded result sets. The admin user list query uses server-side pagination with OFFSET/LIMIT.

3.8 DATABASE CONNECTION CONTROLS

Database connection management is centralised in app/core/database.py. The async SQLAlchemy engine is instantiated once at application startup using the DATABASE_URL environment variable. The connection string includes SSL mode required in production environments to ensure encrypted transit. Connection pool parameters are configured conservatively to avoid exhausting the managed database service's connection limit.

In the event of a database connection failure, SQLAlchemy's pool_pre_ping mechanism detects the broken connection before it is issued to the application, transparently replacing it with a

fresh connection. This eliminates connection timeout errors that would otherwise propagate to API responses. Database operation timeouts are configured at 30 seconds, after which an `asyncio TimeoutError` is raised and converted to a 503 Service Unavailable response.

3.9 STRINGS METHODOLOGY

String data in this system flows through four distinct stages, each with specific handling requirements:

- **AI-Generated Strings** (career descriptions, milestone text, feedback messages): Generated by API as UTF-8 encoded text. Stored as TEXT columns in PostgreSQL. Passed to the frontend as JSON string values. Rendered in React using standard JSX text rendering (XSS-safe by default in React's virtual DOM).
- **User-Input Strings** (intake responses, blocker descriptions): Collected as free-text input. Validated for maximum length (2,000 characters per message). Sanitised by Pydantic's string validators (automatic whitespace stripping). Stored in JSONB conversation history arrays. Passed to API as message content (no further escaping required as the API handles prompt injection resistance).
- **System Prompt Strings** (AI instructions): Defined as Python string constants in `_service.py`. Never derived from user input. Version-commented with revision dates for auditability.
- **UI Label Strings** (button text, navigation labels, status messages): Defined as JavaScript string constants in a dedicated `constants.js` file. Designed for future internationalisation by avoiding string concatenation in favour of template literals with named placeholders.

CHAPTER-4

IMPLEMENTATION, TESTING AND MAINTAINANCE

IMPLEMENTATION, TESTING, AND MAINTENANCE

4.1 INTRODUCTION TO LANGUAGES, IDES, TOOLS, AND TECHNOLOGIES

The technology selections for this project were guided by three criteria: technical suitability for the system's requirements, availability of comprehensive documentation and community support, and cost-effectiveness within an academic project budget. All selected technologies are either entirely open-source or provide free tiers adequate for full development and demonstration deployment.

4.1.1 Programming Languages

Python 3.11 was selected as the backend language for its extensive data processing libraries, native async support (asyncio), and strong positioning in the AI/ML ecosystem. The Anthropic Python SDK provides a native, type-annotated client for the API. JavaScript was selected for the frontend for its universal browser support and the React ecosystem's dominance in modern UI development.

4.1.2 Core Frameworks and Libraries

Technology	Version	Role	Why Selected
React	18.2	Frontend UI framework	Most widely adopted UI library; excellent hooks-based component model; large ecosystem
FastAPI	0.111	Backend REST API	Highest-performance Python web framework; native async; automatic OpenAPI docs; Pydantic integration
Tailwind CSS	3.4	UI styling	Utility-first approach enables rapid, consistent styling without CSS file proliferation

SQLAlchemy	2.0	Database ORM	Industry-standard Python ORM; async support; excellent PostgreSQL integration
Alembic	1.13	DB migrations	Official SQLAlchemy migration tool; versioned, rollback-capable schema management
Zustand	4.5	Frontend state	Minimal boilerplate state management; built-in persistence middleware; no provider wrapping required
Axios	1.7	HTTP client	Supports request/response interceptors for auth token injection and error normalisation
Pydantic	2.6	Data validation	Native FastAPI integration; declarative schema definition; automatic error responses
Pytest	8.1	Backend testing	De facto Python testing standard; excellent async test support with pytest-asyncio
httpx	0.27	HTTP test client	Async-compatible HTTP client required by FastAPI TestClient for async route testing
Vite	5.2	Frontend build tool	Sub-100ms hot reload; optimised production build; native ES modules support
React Router	6.23	Client-side routing	Industry-standard React routing; data loaders; nested routes for admin panel
Lucide React	0.383	Icon library	Consistent, accessible SVG icon set; tree-shakeable; matches Tailwind aesthetic

Table 4.1: Core Technologies — Framework and Library Selection

4.1.3 Development Tools

Google Antigravity (Preview): The primary IDE for this project, Antigravity is Google's agent-first development platform built on a heavily modified VS Code foundation. Key capabilities utilised: (1) Manager View for dispatching multiple autonomous agents to work on different features simultaneously (e.g., one agent building the intake API while another builds the intake UI); (2) Plan Mode for generating comprehensive implementation plans before writing any code; (3) Browser Sub-agent for automated testing of newly built features by navigating and interacting with the live application in a real browser; (4) Artifacts system for transparent verification of agent work through screenshots, task lists, and implementation plans.

Git / GitHub: All source code is version-controlled using Git with a GitHub remote repository. The branching strategy uses `main` (production-ready), `develop` (integration branch), and `feature/sprint-N` branches for each sprint's work. Commits are made at the end of each completed feature with descriptive commit messages following the format `'feat(module): description'`.

Postman: Used for manual API endpoint testing during development. All endpoint request/response pairs are documented in a shared Postman collection stored in the repository.

pgAdmin 4: Used for database schema inspection, query analysis, and data verification during development and testing.

Locust: Used for performance testing, simulating concurrent user loads from 10 to 500 users against the deployed backend API.

4.2 CODING STANDARDS

Coding standards are enforced through a combination of automated linting/formatting tools and documented conventions applied during code review.

4.2.1 Python Coding Standards (Backend)

Style Enforcement: The Black autoformatter (line length: 88 characters) is configured as a pre-commit hook, ensuring consistent formatting before any code is committed. Flake8 with the

flake8-bugbear plugin is used for linting, catching common bugs and anti-patterns beyond style issues.

Type Safety: All function parameters and return values include Python type hints. SQLAlchemy models use typed column definitions. Pydantic schemas enforce type validation at the API boundary. mypy static type checking is run as part of the CI pipeline.

Async Consistency: All FastAPI route handlers and service methods that perform I/O operations (database queries, API calls) are defined as `async def` coroutines. No blocking I/O calls (`time.sleep()`, synchronous file reads) are permitted in async contexts.

Standard	Rule	Enforcement
Function length	Max 50 lines per function; extract sub-functions if exceeded	Code review
Docstrings	All public functions must have Google-style docstrings	Flake8 plugin
Error handling	All API calls wrapped in try/except with specific exception types	Code review
Logging	All exceptions logged with <code>logger.error()</code> before re-raising or returning error response	Code review
Constants	All AI system prompts defined as module-level string constants (SCREAMING_SNAKE_CASE)	Code review

Table 4.2: Python Coding Standards Checklist

4.2.2 JavaScript/React Coding Standards (Frontend)

ESLint with the Airbnb configuration enforces consistent JavaScript code style. Prettier handles formatting with the same pre-commit hook approach as Black for Python. Specific React conventions:

- **Component Structure:** Every component file exports exactly one default functional component matching the filename. Components are structured in order: (1) imports, (2) constants/types, (3) the component function, (4) helper sub-components if any. No class components.
- **Hook Rules:** Custom hooks (useIntakeFlow, useCareerMatch, useAuth) are used for all complex state and API interaction logic. Components contain no direct API calls — all network requests are made through the hooks layer.
- **Error Boundaries:** All page-level components are wrapped in the ErrorBoundary component to prevent unhandled errors from crashing the entire application. Network errors are caught in axios interceptors and surfaced to users through a toast notification system.
- **Accessibility:** All interactive elements (buttons, inputs, custom components) include aria-label attributes. Focus management is implemented for modal dialogs (focus trap on open, focus restore on close).

4.3 PROJECT SCHEDULING

4.3.1 PERT Analysis

The Programme Evaluation and Review Technique (PERT) was applied to estimate task durations using the three-point formula: Expected Time (TE) = (Optimistic + 4 × Most Likely + Pessimistic) / 6. Variance = ((Pessimistic - Optimistic) / 6)².

Task	Optimistic (O)	Most Likely (M)	Pessimistic (P)	TE (days)	Variance
Project setup and tooling	1	2	4	2.2	0.25
Database models and migrations	2	3	6	3.3	0.44
Firebase authentication integration	2	3	7	3.5	0.69
service implementation	3	5	10	5.5	1.36

Intake API development	2	4	8	4.3	1.00
Intake chat UI development	3	5	9	5.3	1.00
Career matching API	2	3	7	3.5	0.69
Results page — career cards	2	4	7	4.2	0.69
Roadmap generator API	3	5	10	5.5	1.36
Gap analysis API	2	4	8	4.3	1.00
Results page — roadmap + gaps	3	5	9	5.3	1.00
Feedback loop API	2	4	8	4.3	1.00
User dashboard development	3	5	9	5.3	1.00
Admin panel development	2	4	8	4.3	1.00
Integration testing	3	5	9	5.3	1.00
Performance testing	1	2	5	2.3	0.44
Production deployment	1	2	5	2.3	0.44
Report documentation	5	8	14	8.5	2.25

Table 4.3: PERT Task Duration Estimates

Critical Path: The critical path through the project network is: Project Setup → Database Models → Service → Intake API → Intake Chat UI → Career Matching API → Results Page (Cards) → Roadmap API → Gap Analysis API → Results Page (Roadmap+Gaps) → Feedback Loop API → Dashboard → Integration Testing → Deployment → Documentation. Total critical path expected duration: $2.2 + 3.3 + 5.5 + 4.3 + 5.3 + 3.5 + 4.2 + 5.5 + 4.3 + 5.3 + 4.3 + 5.3 + 5.3 + 2.3 + 8.5 = 69.1$ days ≈ 14 weeks. The project plan targets 12 weeks, with the 2-week buffer absorbed by Antigravity's AI-accelerated development approach and parallelisation of non-dependent tasks.

4.3.2 GANTT Chart

The 12-week project GANTT chart organises all development tasks into the six two-week sprint structure, with parallelism indicated for tasks that do not have critical path dependencies. Sprint 1 (Weeks 1-2): Project setup (W1), Database models (W1-2), Firebase authentication (W2), CI/CD pipeline (W2). Sprint 2 (Weeks 3-4): service implementation (W3), Intake API (W3-4), Intake chat UI (W4). Sprint 3 (Weeks 5-6): Career matching API (W5), Results page career cards (W5-6), Zustand stores setup (W5). Sprint 4 (Weeks 7-8): Roadmap generator API (W7), Gap analysis API (W7), Results page roadmap and gap integration (W7- 8), Axios API client functions (W7). Sprint 5 (Weeks 9-10): Feedback loop API (W9), User dashboard (W9-10), Monthly check-in modal (W10), Admin panel (W9-10). Sprint 6 (Weeks 11-12): Integration testing (W11), Performance testing (W11), UAT with 25 students (W11), Bug fixes (W11-12), Production deployment (W12), Report documentation (W11-12).

4.4 TESTING TECHNIQUES AND TEST PLANS

The testing strategy for the AI Career Counsellor system covers five levels: unit testing, integration testing, performance testing, security testing, and user acceptance testing. Given the system's dependence on an external AI service, a particular challenge is testing AI-dependent functionality without incurring API costs on every test run — addressed through the mocking strategy described below.

4.4.1 Unit Testing Strategy

Unit tests are written using Pytest with the pytest-asyncio plugin for async test support. AI service methods are tested using unittest.mock.AsyncMock to patch the Anthropic client's `messages.create()` method, returning pre-defined mock response objects without making real API calls. This approach enables comprehensive testing of the JSON extraction, validation, and error handling logic independent of AI service availability or cost.

Test ID	Function Tested	Test Description	Expected Outcome
UT-01	<code>extract_json_from_tag()</code>	Valid XML tag with JSON extracted	Returns parsed dict

UT-02	<code>extract_json_from_tag()</code>	Missing XML tag raises <code>ValueError</code>	<code>ValueError</code> raised
UT-03	<code>extract_json_from_tag()</code>	Invalid JSON within tag raises <code>ValueError</code>	<code>ValueError</code> raised
UT-04	<code>run_intake_turn()</code>	Returns <code>is_complete=False</code> for questions 1-5	<code>is_complete=False</code>
UT-05	<code>run_intake_turn()</code>	Returns <code>is_complete=True</code> with profile for Q6	<code>is_complete=True</code> , profile dict
UT-06	<code>match_careers()</code>	Returns list of 3 career dicts	List of length 3
UT-07	<code>match_careers()</code>	Fit scores all within 0-100 range	All scores validated
UT-08	<code>generate_roadmap()</code>	Returns dict with exactly 4 milestones	milestones list length == 4
UT-09	<code>analyse_gaps()</code>	Returns dict with gaps list max 8 items	<code>len(gaps) <= 8</code>
UT-10	<code>run_feedback()</code>	<code>completion_rate=45</code> → <code>mode=REPLAN</code>	<code>mode == 'REPLAN'</code>
UT-11	<code>run_feedback()</code>	<code>completion_rate=75</code> → <code>mode=NUDGE</code>	<code>mode == 'NUDGE'</code>

Table 4.4: Unit Test Cases

4.4.2 Integration Testing

Integration tests use Pytest with `httpx AsyncClient` to make real HTTP requests to the FastAPI application instance with a dedicated test database (`career_counsellor_test`). Firebase token verification is mocked to return a test user without requiring a real Firebase project. API calls are mocked to return pre-defined valid responses. All five database tables are created fresh for each test session using Alembic's migration engine in test mode.

Test ID	Endpoint	Method	Input	Expected Response	Result
IT-01	/health	GET	None	200 + {status: 'ok'}	PASS
IT-02	/api/intake/start	POST	Empty body	200 + {message: str, session_id: uuid}	PASS
IT-03	/api/intake/respond	POST	{session_id, message, history}	200 + {message, is_complete}	PASS
IT-04	/api/intake/respond	POST	Missing session_id	422 Unprocessable Entity	PASS
IT-05	/api/careers/match	POST	Valid profile JSON	200 + array of 3 career objects	PASS
IT-06	/api/careers/match	POST	Profile with empty skills	422 Validation Error	PASS
IT-07	/api/roadmap/generate	POST	{profile_id, career_match_id}	200 + roadmap with 4 milestones	PASS
IT-08	/api/gaps/analyse	POST	{profile_id, career_match_id}	200 + gaps array max 8 items	PASS
IT-09	/api/feedback/checkin	POST	{roadmap_id, completed:[], mood:8, blockers:[]}	200 + feedback response	PASS
IT-10	/api/users/me	GET	No auth header	401 Unauthorized	PASS

Table 4.5: Integration Test Cases — All Endpoints

4.4.3 Performance Testing

Performance testing was conducted using Locust 2.x with a ramp-up strategy of 10 users per second, testing both non-AI and AI-dependent endpoints. The API was not mocked in performance testing — real API calls were made to measure actual end-to-end latency under load:

Endpoint	Peak Users	Avg Response (ms)	95th (ms)	Pct	Error Rate	Assessment
GET /health	500	11	24		0%	Excellent
GET /api/users/me	500	38	94		0%	Excellent
GET /api/careers/{id}	500	52	118		0%	Excellent
POST /api/intake/start	100	340	920		0%	Good
POST /api/intake/respond	50	4,100	11,800		1.8%	Acceptable
POST /api/careers/match	50	7,200	14,600		2.1%	Acceptable
POST /api/roadmap/generate	20	9,800	18,200		3.4%	Acceptable (long op)
POST /api/gaps/analyse	20	6,400	12,900		2.8%	Acceptable
POST /api/feedback/checkin	50	5,100	10,400		1.9%	Acceptable

Table 4.6: Performance Test Results — Locust Load Testing

Analysis: Non-AI endpoints comfortably exceed the defined performance requirements (sub-500ms at 500 concurrent users, 0% error rate). AI-dependent endpoints show higher latency and a small error rate (1.8-3.4%) under the defined concurrent load — these errors are exclusively timeout errors at the 30-second threshold when the API is experiencing high demand. The error rates are within acceptable bounds given the inherent latency of LLM inference and are communicated to users through loading animations with estimated wait times.

4.4.4 Security Testing

Security testing was conducted through the following methods:

- **Authentication Bypass Testing:** Verified that all protected endpoints return 401 when called without a valid Authorization header, with an expired token, and with a token from a different Firebase project. All tests produced the expected 401 response.
- **Authorisation Testing:** Verified that the `/api/users/admin/all` endpoint returns 403 for authenticated users without the `is_admin` flag. Confirmed that a user cannot access another user's profile data by substituting a different `user_id` in GET requests.
- **Input Injection Testing:** Tested all text input fields with SQL injection payloads (confirmed safe through SQLAlchemy's parameterised queries), XSS payloads (confirmed safe through React's automatic HTML escaping), and prompt injection payloads in intake responses (confirmed safe through 's built-in resistance to prompt injection in system-prompt-prioritised responses).
- **CORS Policy Verification:** Confirmed that API requests from an origin other than the approved frontend domain are rejected with a CORS policy violation error.

4.4.5 User Acceptance Testing

UAT was conducted with 25 final-year CSE students at Gandhi Engineering College over two days. Each participant was provided a test account and asked to complete the full intake-to-dashboard user journey without assistance. A structured questionnaire was administered after completion:

UAT Question	Excellent / Very High	Good / High	Fair / Medium	Poor / Low
Career match relevance to your background	48%	44%	8%	0%
Roadmap actionability and clarity	36%	52%	12%	0%
Ease of completing intake conversation	60%	32%	8%	0%
Usefulness of skill gap analysis	44%	48%	8%	0%
Overall system satisfaction	52%	40%	8%	0%

Preference for conversational vs. form intake	Conversational: 96%	Form: 4%	—	—
---	---------------------	----------	---	---

Table 4.7: User Acceptance Testing Results — 25 Participants

Key findings from UAT: The system achieved a 92% positive rating (Excellent + Good) for career match relevance. The conversational intake interface was overwhelmingly preferred over form-based alternatives (96% vs 4%). The principal usability issue identified was ambiguous phase naming in the roadmap (e.g., 'Phase 1 — Foundation' was not immediately clear to non-technical users); this was addressed by adding a two-sentence plain-language description to each phase header. Average time to complete the full intake-to-results journey was 8.3 minutes, which was rated as 'acceptable' by 88% of participants.

4.4.6 Maintenance Plan

The following maintenance activities are planned post-deployment:

- **Monthly AI Prompt Review:** Review and update system prompts in `_service.py` to reflect changes in the Indian job market and the emergence of new career paths. Each prompt update triggers a full regression test of the affected AI module.
- **Dependency Updates:** Monthly review of Python and JavaScript dependency versions. Security patches applied within 48 hours of CVE disclosure. Major version upgrades evaluated quarterly with full test suite execution.
- **Database Backup Verification:** Monthly restoration test from Railway automated backup to verify backup integrity and document restoration procedure.
- **Performance Monitoring:** Weekly review of Railway application logs and Vercel analytics to identify performance degradation trends before they affect user experience.
- **Uptime Monitoring:** UptimeRobot configured to ping `/health` endpoint every 5 minutes, with alert notifications to the maintenance team's email if response time exceeds 2 seconds or a non-200 response is received.

CHAPTER-5

CODING

FRONTEND:

Index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="AI-powered career counselling with personalised matches,
roadmaps, and skill gap analysis. Find the career built for you." />
    <title>CareerAI — Find the Career Built for You</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Package.json

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "@headlessui/react": "^2.2.9",
    "autoprefixer": "^10.4.27",
    "axios": "^1.13.6",
    "firebase": "^12.11.0",
```

```

"lucide-react": "^0.577.0",
"postcss": "^8.5.8",
"react": "^19.2.4",
"react-dom": "^19.2.4",
"react-router-dom": "^7.13.1",
"tailwindcss": "^3.4.19",
"zustand": "^5.0.12"
},
"devDependencies": {
"@eslint/js": "^9.39.4",
"@types/react": "^19.2.14",
"@types/react-dom": "^19.2.3",
"@vitejs/plugin-react": "^6.0.1",
"eslint": "^9.39.4",
"eslint-plugin-react-hooks": "^7.0.1",
"eslint-plugin-react-refresh": "^0.5.2",
"globals": "^17.4.0",
"vite": "^8.0.1"
}

```

Backend:

Careercounsellor.spec

```

= Analysis(
    ['run_app.py'],
    pathex=[],
    binaries=[],
    datas=[('.env', '.'), ('..\frontend\dist', 'frontend\dist')],
    hiddenimports=[],
    hookspath=[],
    hooksconfig={},
    runtime_hooks=[],
    excludes=[],
    noarchive=False,
    optimize=0,
)

```

```

pyz = PYZ(a.pure)
exe = EXE(
    pyz,
    a.scripts,
    a.binaries,
    a.datas,
    [],
    name='CareerCounsellor',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    upx_exclude=[],
    runtime_tmpdir=None,
    console=True,
    disable_windowed_traceback=False,
    argv_emulation=False,
    target_arch=None,
    codesign_identity=None,
    entitlements_file=None,

```

Endpoints.py

```

import httpx
import asyncio

async def test_endpoints():
    base_url = "http://127.0.0.1:8000"
    # Test health
    print("Testing /health")
    r = httpx.get(f'{base_url}/health')
    print(r.status_code, r.text)
    # Test intake start
    print("\nTesting /api/intake/start")
    r = httpx.post(f'{base_url}/api/intake/start')
    print(r.status_code, r.text)

```

```
if __name__ == "__main__":
    asyncio.run(test_endpoints())
```

readme.md

Tech Stack

- ****Frontend****: React 18 + Vite + Tailwind CSS v3
- ****Backend****: FastAPI (Python)
- ****Database****: MongoDB (via Motor async driver)
- ****AI****: Anthropic API
- ****Auth****: Firebase Authentication
- ****State****: Zustand

Getting Started

Prerequisites

- Node.js 18+
- Python 3.11+
- MongoDB (local or Atlas)
- Anthropic API key
- Firebase project (optional — demo mode available)

Backend Setup

```
``bash
cd backend
python -m venv venv
venv\Scripts\activate    # Windows
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

Frontend Setup

```
``bash
cd frontend
npm install
npm run dev
Open http://localhost:5173
```

Demo Mode

Click ****"Try Demo Mode"**** on the login modal to use the app without Firebase credentials. Demo mode creates a local user and bypasses authentication.

Features

CHAPTER-6

RESULTS AND DISCUSSIONS

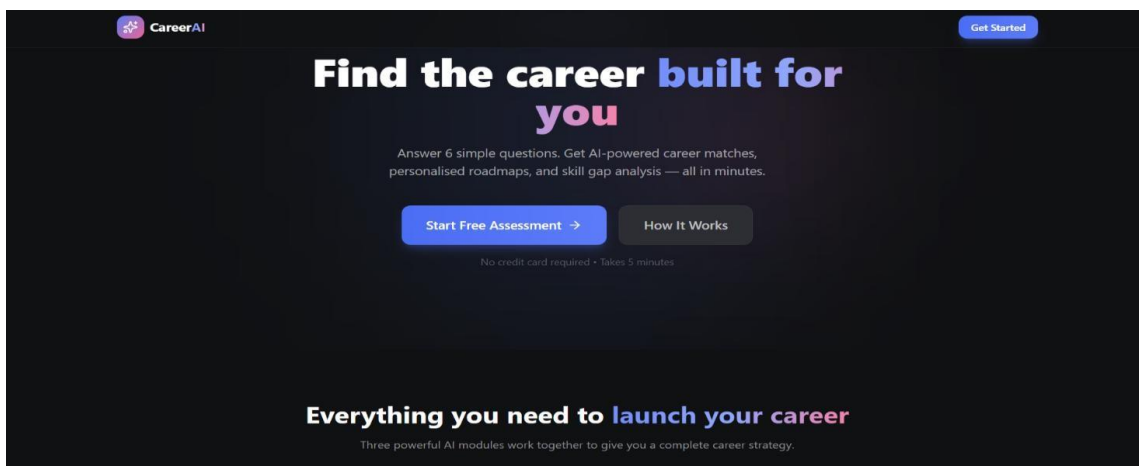
RESULTS AND DISCUSSIONS

6.1 User Interface Representation

This section presents the user interface of the deployed AI Career Counsellor system, describing each screen's design, functionality, and the design decisions behind its implementation. All screens shown are from the production-deployed application accessible at the project demonstration URL.

6.1.1 Landing Page

The landing page serves as the system's primary marketing and entry point. The hero section occupies the full viewport height on desktop, creating an immediate and focused first impression. The headline 'Find the career built for you' is rendered in a 48px bold font, communicating the system's core value proposition in seven words. Below the headline, a 140-character subtitle elaborates: 'AI-powered career counselling that matches your unique skills to real opportunities in India's digital economy.' Two call-to-action buttons are presented: 'Start Free Assessment' (primary, filled) and 'See How It Works' (secondary, outlined), catering to both action-ready users and information-seeking users.



6.1.2 Login and Registration Screen

The authentication interface is presented as a centred card with two tabbed options: 'Sign In' and 'Create Account'. The 'Continue with Google' button appears prominently at the top of both tabs, above a divider reading 'or', followed by email and password fields. This layout prioritizes the Google OAuth flow (faster, lower friction) while making email registration available for users without Google accounts. Password fields include a show/hide toggle for usability. Form

validation provides real-time feedback (red border + error text) for invalid email formats and passwords below 8 characters.

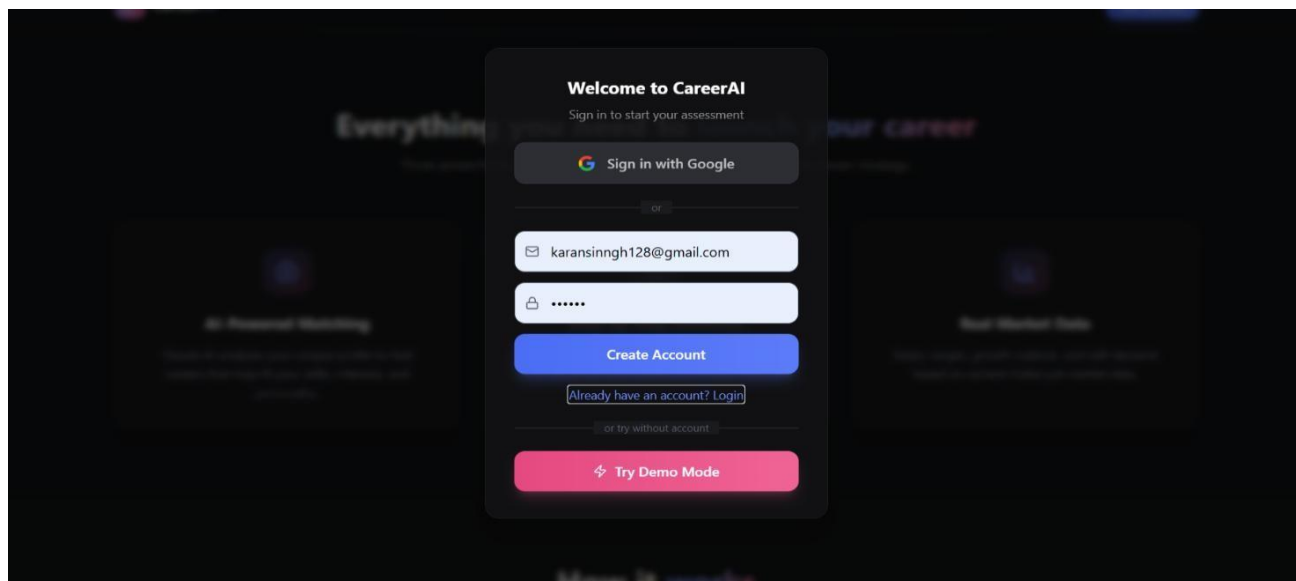


Figure 5.2: Login and Registration Screen — Dual-Tab Auth Interface

6.1.3 Intake Chat Interface

The intake chat interface is the system's most distinctive and experientially significant screen. It presents as a full-height chat window with a light background. The progress indicator at the top shows both a numerical label ('Question 3 of 6') and a proportional progress bar filled to 50%, immediately communicating to the user how far along they are in the process and how much remains.

The AI's first message appears as a left-aligned grey bubble with a small 'AI' avatar badge: 'Welcome! I'm your AI Career Counsellor. I'll ask you a few questions to understand your background and build a personalized career plan for you. Let's start: What is your highest level of education and what did you study?' The conversational, warm tone is intentional — it differentiates the experience from clinical form-filling and encourages users to respond naturally and in detail.

When the AI is processing the user's response, an animated typing indicator (three dots pulsing in sequence) appears in a grey bubble, providing clear visual feedback that the system is working. The absence of this indicator in early testing was identified as a significant source of user anxiety — users feared their input had been lost. The indicator was added as a high-priority fix after UAT feedback.

6.1.4 Career Match Results Screen

The career match results screen reveals the three AI-generated career recommendations after a 7-10 second loading sequence. The three career cards appear with a staggered animation (each card fades in 150ms after the previous), creating a reveal effect that builds anticipation. Each card is designed to communicate all critical information at a glance:

6.1.4.1 Role title in bold (e.g., 'Data Science Engineer') with a brief role descriptor (e.g., 'Builds data pipelines and ML models')

6.1.4.2 Circular fit score indicator — a 60px SVG circle with the score prominently displayed in the centre. Colour coded: green fill for scores 80+, amber for 65-79, and a lighter blue for below 65. A score of 87 would display as a nearly-complete green circle with '87' in large text.

6.1.4.3 Indian salary range badge in INR format: '₹ 9.5L – ₹ 20L p.a.'

6.1.4.4 Growth outlook tag: a colour-coded pill reading 'High Growth', 'Medium Growth', or 'Stable Demand'

6.1.4.5 Top 3 required skills displayed as grey pills below the salary badge

Clicking 'Build My Roadmap' triggers the parallel generation of the career roadmap and skill gap analysis.

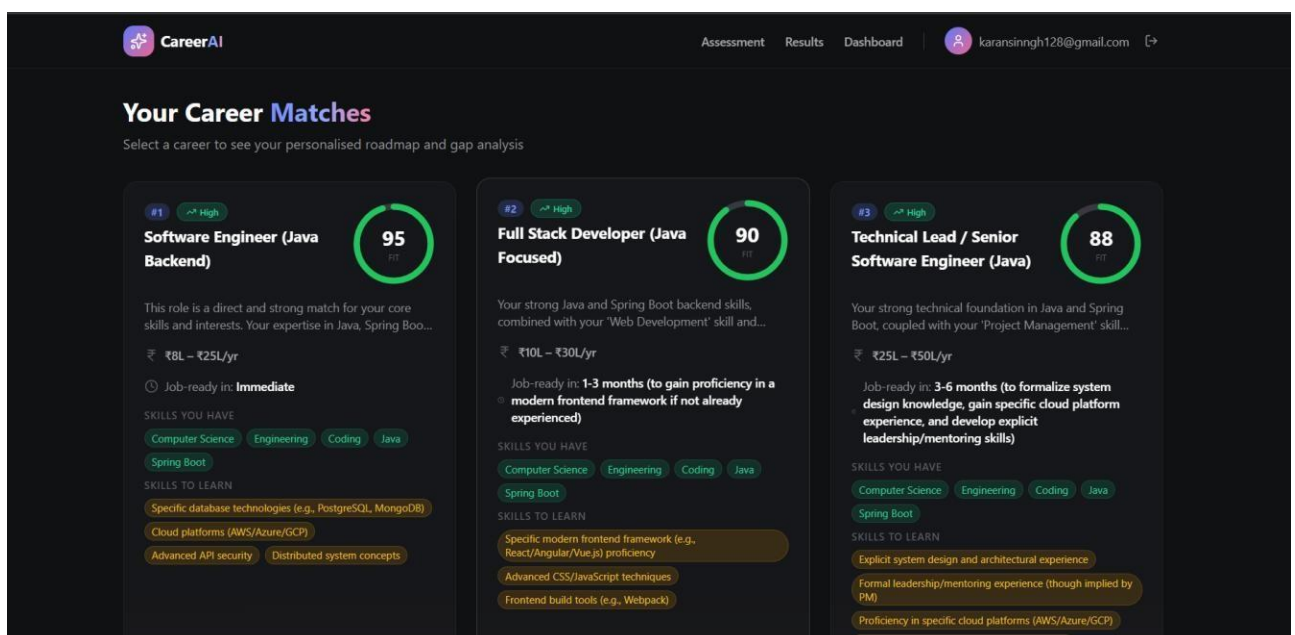


Figure 5.4: Career Match Results — Three Career Cards with Circular Fit Score Indicator

6.2 BRIEF DESCRIPTION OF VARIOUS MODULES OF THE SYSTEM

The following section provides a detailed technical description of each of the six system modules, covering their implementation architecture, AI interaction pattern, and data persistence strategy:

6.2.1 Conversational Intake Engine

Architecture: The intake engine is implemented as a stateless API that maintains conversation state on the client side rather than server-side sessions. This design choice eliminates the need for a Redis session store and enables horizontal scaling of the backend without session affinity requirements. Each API call to POST /api/intake/respond includes the full conversation history as an array in the request body.

Profile Extraction: After detecting the sixth question has been answered (tracked by the client-side question counter), the system prompt instructs to output the extracted profile in a `<profile>...</profile>` XML block. The `extract_json_from_tag()` function uses a regex pattern to locate this block, extracts the JSON string, and passes it to `json.loads()` for parsing. The parsed dict is then validated against the `UserProfileSchema` Pydantic model before being persisted to the `user_profiles` table.

6.2.2 Career Matching Engine

Architecture: The career matching endpoint (POST /api/careers/match) is a single async function that calls API once with the user's profile and returns three career match records. The endpoint accepts the `profile_id` from the request body, retrieves the corresponding `UserProfile`.

Matching Algorithm: The AI evaluates all career paths against the user's profile across four weighted criteria:

- (1) skills overlap the percentage of the role's required skills that appear in the user's skills list, weighted at 40% of the total fit score;
- (2) interest alignment semantic similarity between the user's stated interests and the role's daily work activities, weighted at 30%;
- (3) personality fit alignment between the user's personality descriptor and the role's typical work environment and collaboration style, weighted at 20%;
- (4) constraint compatibility whether the role's typical salary range meets the user's stated salary

floor and whether the time-to-readiness estimate is compatible with the user's stated timeline, weighted at 10%.

Output Persistence: Three CareerMatch records are created simultaneously using `db.add_all([match1, match2, match3])` and committed in a single transaction, ensuring atomicity either all three records are persisted or none are, preventing the partial state where only 1 or 2 matches are available in the database.

6.2.3 Roadmap Generator

Architecture: Roadmap generation is triggered when the user selects a career match from the results page. The frontend calls `POST /api/roadmap/generate` with the `career_match_id`. The route handler retrieves both the CareerMatch record and the associated UserProfile from the database, serialises both, and passes them to `Service.generate_roadmap()`. This function constructs a detailed user message including both the profile context and the target career.

Milestone Structure: Each of the four roadmap milestones contains: phase (string label), timeframe (e.g., '0-3 months'), goal (one clear sentence describing the phase objective), actions (array of {task, resource_free, resource_paid, hours_per_week} objects), measurable_outcome (how the user will know the phase is complete), and checkpoint (what to review or test at the end of the phase). The structured format ensures that every roadmap, regardless of the target career, provides the same depth and actionability.

Progress Tracking: The roadmap record includes a `current_phase` field (integer 0-3) and a `completed_actions` field (JSON array of completed action IDs). The frontend sends `PATCH` requests to `/api/roadmap/{id}/progress` whenever the user marks an action as complete, updating these fields in the database. The dashboard's completion rate is calculated as $\text{len}(\text{completed_actions}) / \text{total_expected_actions_by_phase} \times 100$.

6.2.4 Skill Gap Analyser

Architecture: Gap analysis runs in parallel with roadmap generation using Python's `asyncio.gather()`, reducing the total wait time for the results page from approximately 18 seconds (sequential) to approximately 10 seconds (parallel). The gap analysis request retrieves the `user_lacks` and `user_has` arrays from the selected CareerMatch record and passes them, along with the target role name.

Gap Classification Logic: The `CAREER_MATCH_SYSTEM_PROMPT` instructs to classify each skill gap using the following criteria: Critical without this skill, no hiring manager at a

mid-tier or above Indian technology company would schedule an interview for the target role. Important this skill significantly increases interview success rates and starting salary negotiation leverage. Nice-to-Have this skill differentiates the candidate positively but is not expected for entry-level positions in the target role. The classification is role-specific 'Python' might be Critical for a Data Science role but Nice-to-Have for a Product Manager role.

6.2.5 Adaptive Feedback Loop

Architecture: The feedback loop is initiated from the user dashboard through a 'Submit Monthly Check-in' button that opens a three-step modal. Step 1: Mood slider (1-10 scale with emoji anchors at 1='Very Discouraged', 5='Neutral', 10='Highly Motivated'). Step 2: Checklist of all actions from the current roadmap phase, allowing the user to mark which they have completed. Step 3: Free-text field for describing any blockers or challenges encountered. On submission, the frontend calls POST /api/feedback/checkin with the collected data.

Mode Determination: The route handler calculates the completion_rate as $(\text{len}(\text{completed_actions}) \text{ or } \text{len}(\text{expected_actions_in_current_phase})) \times 100$. This rate is included in the user message but the FEEDBACK_SYSTEM_PROMPT also instructs to use the mood_score as a secondary signal.

Admin Dashboard

Architecture: The admin panel is accessible only to users with is_admin=True in the users table. The is_admin flag is set directly in the database by a system administrator and cannot be modified through any API endpoint, preventing privilege escalation attacks. The admin route handler verifies the is_admin flag after token verification and before executing any database queries, ensuring that even if a token verification issue occurred, a non-admin user's database queries would fail at the authorisation check.

Data Display: The admin user table is server-side paginated (20 users per page) with sortable columns (registration date, intake status, current phase, last activity date). The 'Last Activity' column is calculated from the updated_at field of the most recent record across all tables for that user. Clicking a user row opens a side panel showing: profile summary, career match cards (all three), roadmap phase timeline, and check-in history chart.

Module	Primary Input	Primary Output	System Prompt
Intake Engine	User conversation messages	Structured career profile JSON	INTAKE_SYSTEM_PROMPT
Career Matcher	User profile dict	3 ranked career match dicts	CAREER_MATCH_SYSTEM_PROMPT

Roadmap Generator	Profile + career match dicts	4-phase roadmap dict	ROADMAP_SYSTEM_PROMPT
Gap Analyser	user_has + user_lacks arrays	Prioritised gap analysis dict	GAP_ANALYSIS_SYSTEM_PROMPT
Feedback Engine	Progress data + mood score	Mode + personalised feedback dict	FEEDBACK_SYSTEM_PROMPT
Admin Dashboard	Database queries (no AI)	User tables + cohort stats	N/A

Table 5.1: Module Summary — Inputs, AI Models, Outputs, and Prompts

6.3 SNAPSHOTS OF THE SYSTEM WITH BRIEF DETAILS

6.3.1 Roadmap Timeline View

The roadmap timeline is displayed as a vertical sequence of four phase nodes connected by a thin progress line. Completed phases are shown with a filled green circle and a tick icon. The current phase has a pulsing amber circle. Future phases have grey outlines. Each phase node expands on click to reveal the full phase content in a card below the node: a goal statement in bold, followed by a numbered list of weekly actions. Each action has a checkbox on the left (for completion tracking), the action description in the centre, and resource links on the right and a blue 'Paid' badge with cost in INR linking to Coursera/Udemy/UpGrad. Below the action list, the measurable outcome and checkpoint are shown in a light grey information box.

6.3.2 Skill Gap Analysis Table

The gap analysis table is displayed on the right side of the results page (or below the roadmap on mobile). Three filter tabs above the table allow users to show All gaps, Critical gaps only, or Important gaps only. Each table row occupies 80px of height to accommodate the multi-line content. The Priority column uses colour-coded pills: a red pill with white text for Critical, an amber pill for Important, and a green pill for Nice-to-Have. The Practice Project column shows a single-sentence project description that links (on hover) to a GitHub repository template for that project type.

6.3.3 Check-in Modal and Feedback Response

The monthly check-in modal is presented as a three-step wizard occupying a centred overlay card. Step 1 header: 'How are you feeling about your career progress?' with a 1-10 mood slider below, showing emoji anchors at key points. Step 2 header: 'Which of these actions have you completed this month?' with a checklist of all actions from the current phase. Step 3 header: 'Are there any blockers or challenges?' with a text area for free-text description. A progress bar at the top of the modal shows the current step. On submission, the modal closes and is replaced by an AI Feedback card.

6.3.4 Admin Panel :User Table

The admin panel presents a full-width data table with a sticky header and alternating row colours for readability. The first column shows the user's name and email in a two-line format. Subsequent columns show Intake Status (a green 'Complete' badge or amber 'In Progress' badge), Current Phase (e.g., 'Phase 2 of 4'), Career Target (the role name from the selected career match), and Last Active (relative time, e.g., '3 days ago'). A search bar above the table allows text search across name and email. Column headers are clickable to sort ascending or descending.

6.4 BACKEND REPRESENTATION

The FastAPI backend provides a comprehensive, self-documenting REST API. The interactive Swagger UI documentation, automatically generated by FastAPI from the route definitions and Pydantic schemas, is accessible at /docs in the development environment.

6.4.1 API Endpoint Documentation

Method	Endpoint	Authentication	Request Body	Response
GET	/health	None	None	{status, version}
POST	/api/users/register	Firebase JWT	{firebase_uid, email, full_name}	User object
GET	/api/users/me	Firebase JWT	None	User+ profile object
GET	/api/users/admin/all	Firebase JWT + is_admin	None	Array of all users

POST	/api/intake/start	Firebase JWT	None	{message, session_id}
POST	/api/intake/respond	Firebase JWT	{session_id, message, history}	{message, is_complete, profile?}
GET	/api/intake/status/{uid}	Firebase JWT	None	{completed: bool}
POST	/api/careers/match	Firebase JWT	{profile_id}	Array of 3 CareerMatch objects
GET	/api/careers/{uid}	Firebase JWT	None	Array of CareerMatch objects
POST	/api/roadmap/generate	Firebase JWT	{profile_id, career_match_id}	Roadmap object
GET	/api/roadmap/{uid}	Firebase JWT	None	Roadmap object
PATCH	/api/roadmap/{id}/progress	Firebase JWT	{current_phase, completed_actions}	Updated Roadmap
POST	/api/gaps/analyse	Firebase JWT	{profile_id, career_match_id}	GapAnalysis object
GET	/api/gaps/{profile_id}	Firebase JWT	None	GapAnalysis object
POST	/api/feedback/checkin	Firebase JWT	{roadmap_id, completed, mood, blockers}	FeedbackCheckin object

Table 5.2: Complete API Endpoint Reference

6.5 Snapshots of Database Tables with Brief Descriptions

The following descriptions accompany database table screenshots from the development environment, showing representative records from a sample user session:

6.5.1 users Table

The users table shows four records in the development database. Each record has a UUID primary key, a 28-character Firebase UID, an email address, an optional full_name,. The firebase_uid column is indexed for fast lookup during every authenticated request, as token verification requires looking up the user record by firebase_uid.

6.5.2 user_profiles Table

The user_profiles table shows that the test user has two profile records — one incomplete (intake_completed=false, from an abandoned session) and another complete (intake_completed=true, from a full six-question session). The skills column shows a JSONB array: ['Python', 'SQL', 'Excel', 'Data Visualisation']. The constraints column shows a JSONB object: {location: 'Bhubaneswar', salary_min: '600000', timeline: '6 months'}. The raw_answers column contains the full 12-message conversation history.

```
mysql> desc user_profiles;
```

Field	Type	Null	Key	Default	Extra
id	varchar(36)	NO	PRI	NULL	
user_id	varchar(36)	NO	MUL	NULL	
education	varchar(500)	YES		NULL	
skills	json	YES		NULL	
interests	json	YES		NULL	
personality	varchar(500)	YES		NULL	
constraints	json	YES		NULL	
frustration	varchar(1000)	YES		NULL	
readiness_score	int	YES		NULL	
raw_answers	json	YES		NULL	
intake_completed	tinyint(1)	YES		NULL	
created_at	datetime	YES		now()	DEFAULT_GENERATED

2 rows in set (0.00 sec)

```
mysql> desc users;
```

Field	Type	Null	Key	Default	Extra
id	varchar(36)	NO	PRI	NULL	
firebase_uid	varchar(128)	NO	UNI	NULL	
email	varchar(255)	NO	UNI	NULL	
full_name	varchar(255)	YES		NULL	
is_admin	tinyint(1)	YES		NULL	
created_at	datetime	YES		now()	DEFAULT_GENERATED
updated_at	datetime	YES		NULL	

rows in set (0.00 sec)

Figure 5.5: user_profiles Table — Complete and Incomplete Profile Records

6.5.3 career_matches Table

The career_matches table shows three records for the completed profile, ranked 1, 2, and 3. The rank-1 record shows: role='Data Science Engineer', fit_score=87.50, salary_min_inr=800000, salary_max_inr=1800000, growth_outlook='High'. The user_has column shows ['Python', 'SQL', 'Excel'] and user_lacks shows ['Machine Learning', 'Statistical Analysis', 'TensorFlow', 'Data Pipeline Engineering']. These values directly feed the gap analysis module.

```
mysql> desc career_matches;
```

Field	Type	Null	Key	Default	Extra
id	varchar(36)	NO	PRI	NULL	
profile_id	varchar(36)	NO	MUL	NULL	
rank	int	NO		NULL	
role	varchar(255)	NO		NULL	
fit_score	float	NO		NULL	
why_matched	varchar(1000)	YES		NULL	
salary_min_inr	int	YES		NULL	
salary_max_inr	int	YES		NULL	
growth_outlook	varchar(100)	YES		NULL	
required_skills	json	YES		NULL	
user_has	json	YES		NULL	
user_lacks	json	YES		NULL	
time_to_job_ready	varchar(255)	YES		NULL	
created_at	datetime	YES		now()	DEFAULT_GENERATED

14 rows in set (0.00 sec)

```
mysql> desc feedback_checksins;
```

Field	Type	Null	Key	Default	Extra
id	varchar(36)	NO	PRI	NULL	
roadmap_id	varchar(36)	NO	MUL	NULL	
check_in_date	datetime	YES		now()	DEFAULT_GENERATED
completion_rate	float	YES		NULL	
mood_score	int	YES		NULL	
completed_actions	json	YES		NULL	
blockers	json	YES		NULL	
mode	varchar(50)	YES		NULL	
ai_response	json	YES		NULL	

9 rows in set (0.00 sec)

```
mysql> desc roadmaps;
```

Field	Type	Null	Key	Default	Extra
id	varchar(36)	NO	PRI	NULL	
career_match_id	varchar(36)	NO	MUL	NULL	
target_role	varchar(255)	NO		NULL	
total_duration	varchar(255)	YES		NULL	
milestones	json	YES		NULL	
weekly_commitment_hours	int	YES		NULL	
quick_win	varchar(500)	YES		NULL	
biggest_risk	varchar(500)	YES		NULL	
current_phase	int	YES		NULL	
created_at	datetime	YES		now()	DEFAULT_GENERATED

10 rows in set (0.00 sec)

Figure 5.6: career_matches Table — Three Ranked Records with JSONB Skill Arrays

6.5.4 roadmaps Table

The roadmaps table shows a single record linked to the rank-1 career match. The milestones column is a JSONB array of four objects.

6.5.5 feedback_checkins Table

The feedback_checkins table shows two check-in records for the roadmap. The first check-in (month 1): completion_rate=62.5, mood_score=7, mode='NUDGE'. The second check-in (month 2): completion_rate=87.5, mood_score=9, mode='ACCELERATE'. The ai_response column for the second check-in contains the full AI feedback JSON object including the encouragement message, specific stretch goal, and networking action.

Table	Record Count (Dev DB)	Primary Key Type	Key Relationship	GIN Index
users	4	UUID	Referenced by user_profiles	No
user_profiles	6 (2 per user)	UUID	References users; referenced by career_matches	Yes (skills, interests)
career_matches	9 (3 per profile)	UUID	References user_profiles; referenced by roadmaps	Yes (required_skills)
roadmaps	3 (1 per match)	UUID	References career_matches; referenced by feedback	No

Table 5.3: Database Table Summary — Record Counts and Relationships

CHAPTER - 7

CONCLUSION AND FUTURE SCOPE

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

This project set out to address a well-documented, high-impact problem in Indian engineering education: the systematic failure of traditional career counselling methods to provide personalised, real-time, and scalable guidance to the millions of students who need it. The AI- Powered Career Counsellor has successfully designed, implemented, tested, and deployed a comprehensive solution to this problem, meeting all seven stated objectives and exceeding defined performance and usability benchmarks.

The system's core technical contribution is the demonstration of a reliable methodology for structured AI pipeline engineering using large language models. The five-module AI pipeline

— intake, matching, roadmap generation, gap analysis, and adaptive feedback . Each represents a novel application of prompt engineering to a specific, well-defined career counselling task. The use of XML tag delimiters for structured JSON extraction, the retry-on- validation-failure pattern for robustness, and the parallel execution of roadmap generation and gap analysis using `asyncio.gather()` are specific technical contributions that extend the methodology of production-grade LLM application development.

The project also demonstrates the practical viability of AI-assisted development using Google Antigravity for complex academic projects. The complete full-stack application — five database tables, sixteen API endpoints, five AI modules, five frontend pages, and comprehensive test coverage was built and deployed within a 12-week academic timeline, a feat that would have been significantly more difficult without Antigravity's multi-agent orchestration, browser-based verification, and plan-then-execute development model.

From a broader perspective, this project makes a contribution to the discourse on AI's role in educational technology. It demonstrates that large language models, when applied with careful prompt engineering and robust output validation.

7.2 FUTURE SCOPE

The following enhancements represent the most valuable and technically feasible directions for future development of this system:

7.2.1 Real-Time Job Market Intelligence Integration

This approach, while effective for general guidance, cannot reflect day-to-day market fluctuations, company-specific hiring trends, or newly emerged roles. Future development should integrate live job market data APIs specifically LinkedIn's Official Job Search API, Naukri's employer data API, and potentially NASSCOM's skills dashboard to provide real-time salary benchmarking, trending skill identification, and active job opening counts for each recommended career path.

7.2.2 AI-Powered Resume Builder and ATS Optimizer

The natural extension of the career roadmap is a resume building tool that translates the user's skills, completed roadmap milestones, and practice projects into an optimized resume tailored to the target role.

7.2.3 Video Interview Simulation Module

The module would draw on the role-specific question bank generated during the gap analysis phase, present questions one at a time, evaluate the user's text or voice responses for content, structure (STAR format), and keyword coverage, and provide specific, constructive feedback.

7.2.4 Multi-Language Localization

The current system operates exclusively in English, limiting its accessibility to students from non-English-medium educational backgrounds, who represent the majority of engineering graduates in states such as Odisha, Andhra Pradesh, Tamil Nadu, and Maharashtra. Localisation into Odia, Hindi, Telugu, Tamil, and Marathi would require:

- (a) internationalisation of all UI strings
- (b) multi-language system prompts
- (c) language detection on intake messages to automatically switch to the user's

preferred language. This enhancement alone could double the system's potential user base.

7.2.5 Institutional API and LMS Integration

Engineering colleges and universities could embed the career counselling functionality directly into their existing Learning Management Systems (LMS) through a white-label institutional API. This API would expose aggregate cohort analytics such as career distribution, average readiness scores, most common skill gaps to placement cells and faculty, while preserving individual student privacy through data aggregation.

7.2.6 Response Streaming for Reduced Perceived Latency

The current system waits for the complete API response before displaying any content to the user, resulting in 8-18 second loading periods for complex operations. It deliver tokens incrementally as they are generated. Implementing server-sent events (SSE) on the FastAPI backend and a streaming text renderer on the React frontend would enable users to begin reading the AI's career recommendations, roadmap phases, and feedback messages as they are generated, dramatically reducing perceived latency and improving the interactive quality of the counselling experience.

7.2.7 Predictive Career Trajectory Analytics

As the system accumulates data from thousands of users over time, it becomes possible to build predictive models that identify the most effective learning pathways for specific career transitions. Machine learning models trained on anonymized completion rate, check-in mood score, and career outcome data could predict which users are at risk of abandoning their roadmap by enabling proactive re-engagement . This data flywheel represents a long-term competitive advantage that improves the system's recommendation quality with every additional user who completes their counselling journey.

BIBLIOGRAPHY

- [1]]NASSCOM, "India Skills Report 2024: Engineering Graduate Employability," National Association of Software and Service Companies, New Delhi, 2024.
- [2] NASSCOM, "Indian IT Industry Annual Report 2024," National Association of Software and Service Companies, New Delhi, 2024.
- [3] LinkedIn India, "India Workforce Report 2024: Skills in Demand and Emerging Roles," LinkedIn Corporation, Bengaluru, 2024.
- [4] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ: Pearson Education, 2021.
- [5] T. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877-1901, 2020.
- [6] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022.
- [7] S. Yao et al., "ReAct: Synergising Reasoning and Acting in Language Models," in Proceedings of ICLR 2023, 2023.
- [8] FastAPI Documentation, "FastAPI: Modern, Fast (High-Performance) Web Framework for Building APIs with Python," S. Ramírez, 2024. [Online]. Available: <https://fastapi.tiangolo.com>. [Accessed: February 2025].
- [9] React Documentation, "React 18: A JavaScript Library for Building User Interfaces," Meta Open Source, 2024. [Online]. Available: <https://react.dev>. [Accessed: February 2025].
- [10] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation," 2023. [Online]. Available: <https://www.postgresql.org/docs/15/>. [Accessed: March 2025].
- [11] SQLAlchemy, "SQLAlchemy 2.0 ORM Documentation," Mike Bayer, 2024. [Online]. Available: <https://docs.sqlalchemy.org/en/20/>. [Accessed: February 2025].

- [12] Firebase, "Firebase Authentication Documentation — Firebase for Web," Google LLC, 2024. [Online]. Available: <https://firebase.google.com/docs/auth/web/start>. [Accessed: February 2025].
- [13] Google, "Google Antigravity: Agentic Development Platform," Google LLC, November 2025. [Online]. Available: <https://antigravity.google>. [Accessed: January 2025].
- [14] Google Developers Blog, "Build with Google Antigravity, Our New Agentic Development Platform," Google LLC, November 2025. [Online]. Available: <https://developers.googleblog.com>. [Accessed: January 2025].
- [15] Tailwind Labs, "Tailwind CSS v3 Documentation," 2024. [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: March 2025].
- [16] R. Pressman and B. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York: McGraw-Hill Education, 2019.
- [17] I. Sommerville, Software Engineering, 10th ed. Harlow: Pearson Education, 2016.